Kingdom of Saudi Arabia
Ministry of Higher Education
Princess Noura bint Abdulrahman University
College of Computer and Information Sciences
Graduation Project II

المملكة العربية السعودية
وزارة التعليم العالي
جامعة الأميرة نورة بنت عبدالرحمن
كلية علوم الحاسب والمعلومات
مشروع تخرج 2

DRIVER DROWSINESS DEDTECTION PROTOTYPE

By

Student #1 Fay Almutairi

Student #2 Renad Alrashdi

Student #3 Sarah Albassam

Student #4 Rahaf Alhawiti

Student #5 Shahad Otay

Supervised By

Dr. Najwan Abdel Samee

A Graduation Project Report Submitted to
College of Computer Sciences and Information at PNU
in Partial Fulfillment of the Requirements for the
Degree of
Bachelor of Science
in
Information Technology

CCIS, PNU

Riyadh, KSA

1446H

Table of Contents

List of Tables iii

Table of figures iii

List of Symbols and Abbreviations iv

Abstract vi

Chapter 1: Introduction 1

1.Introduction 2

1.1 Problem statement & significance 2

1.2 Proposed solution (System) 2

Aim & Goals: 3

1.3 Project domain & limitationDomain: 4

Limitations & Challenges: 4

Chapter 2: background information & related work 5

2. Background Information and Related work 6

2.1 Background Information 6

2.1.1 AI 6

2.1.2 Edge Computing 8

2.2 Related Work Survey 10

2.2.1. An Intelligent System for Preventing Accidents Due to Driver distraction 10

2.2.2. Real-Time Driver Drowsiness Detection using Computer Vision 11

2.2.3. Evaluation of driver fatigue on two channels of EEG data 12

2.2.4. Driver Drowsiness Detection 13

2.3 Proposed & Similar Systems Comparison 15

Chapter 3: System Analysis 20

3. System Analysis 21

3.1 Requirement Specifications 21

3.1.1 Survey 21

3.2 Requirement Analysis 23
3.2.1 Use case 23
3.2.2 Software Requirements 23
3.2.3 Hardware Requirements 31
3.2.4 Functional and Non-functional Requirements 32
Chapter 4: System Design 33
4. System Design 34
4.1 System Design 34
4.1.1 System architecture 34
4.1.2. System Prototype and Activity Diagram 35
4.1.3. System Flowchart 37
4.1.4 User interface design 38
Chapter 5: Implementation 39
5. Implementation 40
5.1. Implementation Requirements 40
5.1.1 Hardware Requirements 40
5.1.2 Software Requirements 41
5.2. Visual Studio Code configuration 42
5.2.1 Model Code Configuration 44
5.3 Edge Impulse Configuration 47
5.4 Arduino IDE Configuration 51
5.4.1 Arduino Code 52
5.5 The Proposed Architecture 53
Chapter 6: Testing 55
6. Testing 56
6.1 Test Plan 56
6.2 Test cases 57
6.2.1 Case 1 – System Initialization and Model Uploading 57

6.2.2 Case 2 - Normal Eye Blinking 57
6.2.3 Case 3 - Drowsiness Detection in Different Lighting Conditions 58
6.2.4 Case 4 – Detection When No Face in Frame 59
6.3 Test Result 59
6.3.1 Performance Metrics 59
6.3.2 Binary classification 63
Chapter 7: Conclusion 67
7. Conclusion 68
7.1 Evaluation 68
7.2 Future Works 69
References 70
Appendices 73
Appendix A: Survey Questions 73

List of Tables

Table 2-1 Proposed & Similar systems Comparison 19
Table 3-1 Hardware Requirements 31
Table5-1: Hardware Component 41
Table5-2: Software Requirements 42
Table5-3: Learning Parameters 44
Table 5-4: Essential Libraries used 45
Table 6-1: Case 1 – System Initialization and Model Uploading. 57
Table 6-2: Case 2 - Normal Eye Blinking. 58
Table 6-3: Case 3 - Drowsiness Detection in Different Lighting Conditions. 58
Table 6-4: Case 4 – Detection When No Face in Frame 59
Table 6-5: Binary classification system for driver drowsiness detection in many iterations 63
Table 6-6: model's performance 66

Table of figures

Figure 2-1 Block Diagram of MobileNETV2 8
Figure 2-3 Nicla Vision Camera 9
Figure 3-1 Survey analysis 22
Figure 3-2 Use case Diagram 23
Figure 3-3 Arduino IDE Interface 24
Figure 3-4 Arduino IDE Arduino IDE text editor 25

Figure 3-5 Arduino IDE Download 25
Figure 3-6 Arduino IDE instructions for installation and launching. 25
Figure 3-8 Edge Impulse Sign-Up Page 27
Figure 3-9 Edge Impulse Data Acquisition Interface 27
Figure 3-10 Dataset Train/Test Split in Edge Impulse 28
Figure 3-11 Create Impulse Interface in Edge Impulse 28
Figure 3-12 Transfer Learning Settings in Edge Impulse 28
Figure 3-13 Deployment Options in Edge Impulse 29
Figure 3-14 Visual Studio Code interface 30
Figure 3-15 Visual Studio Code extensions 30
Figure 3-16 Visual Studio Code program editor 31
Figure 3-17 Nicla Vision 32
Figure 4-1 system architecture 34
Figure 4-2 System Prototype Modules 35
Figure 4-3 Activity Diagram 36
Figure 4-4 Flowchart 37
Figure 5-1: Python Code for Configuration and Directory Setup. 45
Figure 5-2: Data Preprocessing, Splitting, and Augmentation. 45
Figure 5-3: MobileNetV2-Based Model Architecture 45
Figure 5-4: Evaluation Function for Model Performance Metrics and Confusion Matrix. 46
Figure 5-5: Model Conversion to TensorFlow Lite (TFLite) with Quantization. 46
Figure 5-6 Model Compilation, Training, and Export to TensorFlow Lite. 47
Figure 5-7 Epoch Loop Execution and Training Results Output. 47
Figure 5-8: Uploading the model 48
Figure 5-9 Model saved. 48
Figure 5-10: the model .zip file. 49
Figure 5-11 Edge Impulse CLI. 49
Figure 5-12: Nicla vision firmware. 49
Figure 5-13: Camera connection to the edge impulse. 50
Figure 5-14: Convert the model format. 50
Figure 5-15: Running the AI Model. 51
Figure5-16: The Output result. 51
Figure 5-17: Arduino IDE installation. 51
Figure 5-18: Arduino IDE Interface. 52
Figure 5-19 Initial Configuration of Buzzer Pin and Detection Parameters 52
Figure 5-20 Buzzer Pin Setup in the Arduino setup() Function 53
Figure 5-21 Drowsiness Detection Logic and Buzzer Activation Code 53
Figure 5-22 The proposed architecture. 54
Figure6-1: Showing Confusion Matrix in ML Binary 62

List of Symbols and Abbreviations

PNU	Princess Nora bint Abdul Rahman University
AI	Artificial intelligence
ML	Machine Learning
CNN	Convolutional neural network
UI	User Interface
IDE	Integrated Development Environment

TinyML	Tiny Machine Learning
APIs	Application Programming Interface
DL	Deep Learning
NLP	natural language processing
IoT	Internet of Things
EGG	Electroencephalogram
USB	Universal Serial Bus
LED	light emitting diode
GPS	Global Positioning System

Abstract

Drowsiness while driving is a major cause of car accidents. In the past year, 20% of Saudis reported falling asleep while driving. A study found that about 10% of drivers in Saudi Arabia have either been in or nearly had a car accident due to drowsiness in the past year. Additionally, 20% of the research samples admitted to having previously fallen asleep while driving at least once in the past year. Our graduation project focuses on developing an Artificial Intelligence AI-based Driver Drowsiness Detection System to improve road safety by monitoring driver alertness. Using advanced artificial intelligence techniques, the system combines an Arduino microcontroller with a Nicla Vision camera for real-time drowsiness detection. The Nicla Vision camera captures high-resolution images of the driver's face, and AI algorithms analyze these images to detect signs of drowsiness, such as eye closure. The Arduino platform manages the system's components, processes sensor data, and triggers alerts when drowsiness is detected. This project aims to tackle the critical issue of driver drowsiness, a significant factor in road accidents, by offering a reliable and effective solution for real-time drowsiness detection and mitigation. The system is designed for practical use, robustness, and easy integration into existing vehicle frameworks, highlighting its potential to enhance traffic safety and driver well-being.

Chapter 1: Introduction

1.1. **Problem statement & significance**

1.2. **Proposed solution (System)**

1.3. **Project domain & limitation**

1. Introduction

1.1 Problem statement & significance

Driver drowsiness is one of the leading causes of traffic accidents globally [1], posing a significant risk to both drivers and passengers. When a driver becomes drowsy, their ability to react quickly, maintain focus, and make sound decisions is severely compromised. It can cause a driver to lose control of the vehicle. Drowsy driving is particularly dangerous because it often goes unnoticed by the driver until it is too late. As a result, driver drowsiness poses a serious risk to road safety, this requires innovative solutions that can detect and address driver drowsiness in real time.

In Saudi Arabia, this issue is especially concerning. According to recent studies, 20% of Saudis reported falling asleep while driving in the past year, and 10% of drivers have either been involved in or narrowly avoided accidents caused by drowsiness [2]. These alarming statistics highlight the urgent need to address the problem on a national level. Despite many advancements in vehicle technology, the lack of widespread adoption of drowsiness detection systems leaves a critical gap in ensuring driver safety.

Existing systems often fail to provide real-time feedback or are too complicated to implement on a mass scale [3]. There is an urgent need for a reliable, and easy-to-integrate solution that can actively monitor driver alertness and provide timely intervention to prevent accidents caused by drowsiness.

1.2 Proposed solution (System)

To address the critical issue of driver drowsiness and its significant contribution to road accidents, we propose the development of an AI-based Driver Drowsiness Detection System. This system leverages AI and embedded systems to continuously monitor driver alertness in real-time, offering a proactive solution to enhance road safety by focusing on the early detection of drowsiness [4]. The system's core components include a Nicla Vision camera, which capture high-resolution images of the driver's face. These images will be utilized to train a machine learning model (ML) to detect key signs of drowsiness such as, eye closure [4]. Using the Nicla Vision camera, we are introducing the AI model that is

lightweight and optimized for edge computing. This ensures the system operates independently of internet connections or external processing, making it compact and easily integrated into vehicles [5]. The Arduino microcontroller manages the overall system, processing data from the camera and triggering alerts when drowsiness is detected. When the system identifies signs of drowsiness, an alert is triggered immediately to warn the driver—this could be an audible sound. This real-time feedback helps drivers take necessary action before it leads to an accident, making it especially beneficial on long trips. One of the system's key advantages is its ability to operate without external networks, ensuring reliable performance even in areas with limited connectivity [5]. The cost-effective design and simplicity of the components make it an accessible solution for improving road safety [6].

In conclusion, this AI-based Driver Drowsiness Detection System offers a practical and efficient method to combat driver drowsiness. Its integration of AI and embedded technologies provides a reliable tool for reducing accidents and promoting driver safety.

Aim & Goals:

1. The main goal is to ensure driver safety and minimize a major factor contributing to road accidents.
2. Utilizing the AI to accurately recognize signs of drowsiness. This means keeping a watch on important signs such eye closure.
3. Develop a prototype for the detection of drowsiness with a focus on real-world applicability.
4. The system will promptly alert the driver through sound if it detects signs of drowsiness, ensuring quick action to avoid accidents.

1.3 Project domain & limitation

Domain:

This project aims to develop a real-time driver drowsiness detection prototype that enhances road safety through AI. The system monitors driver alertness by tracking eye patterns using continuous image sequences captured by a Nicla Vision camera. The AI model analyzes these images to determine whether the driver's eyes are open or closed. If the eyes remain closed for a duration indicative of drowsiness, an audio alert is triggered to prevent sleep at the wheel. Utilizing a Tiny machine learning (TinyML) model, the AI classifies eye states based on pre-trained datasets to ensure precise detection. The system operates efficiently and provides a dependable safety solution.

Limitations & Challenges:

- Lack of sufficient amounts of data: The system requires a diverse dataset to accurately detect drowsiness.
- Installation: Professional installation may be needed, which can be costly.
- Maintenance: The system’s components may require maintenance and replacement.
- High dependency on image quality: The effectiveness of the system is heavily reliant on the quality of the images captured by the camera.

Chapter 2: background information & related work

2.1. Background information

2.1.1. AI

2.1.2. Edge computing

2.1.2.1 Arduino Nicla Vision Camera

2.2. Related work survey

2.1.1. An Intelligent System for Preventing Accidents Due to Driver distraction

2.1.2. Real-Time Driver Drowsiness Detection using Computer Vision

2.1.3. Evaluation of driver fatigue on two channels of EEG data

2.1.4. Driver Drowsiness Detection

2.2. Proposed & similar systems comparison

2. Background Information and Related work

2.1 Background Information

The purpose of this project is to develop a real-time driver drowsiness detection system using the Arduino Nicla Vision Camera. Drowsy driving is a significant cause of accidents [1], and this system aims to increase road safety by detecting early signs of driver drowsiness, such as closed eyes, and triggering sound alerts using a buzzer. The Nicla Vision Camera provides built-in image processing capabilities and an Application Programming Interface (API) to support the deployment of advanced Deep Learning (DL) models. It enables the system to process visual data directly on the edge device, ensuring quick detection and alert generation without relying on cloud-based services, which makes the system fast, efficient, and reliable even in areas with limited connectivity.

2.1.1 AI

AI plays a critical role in shaping modern technology. AI is the broader concept of machines being able to carry out tasks in a way that we would consider "intelligent." It encompasses various domains such as Machine Learning (ML), natural language processing (NLP), vision, robotics, and decision-making. AI systems mimic cognitive functions like learning and problem-solving, aiming to replicate human-like intelligence. On the other hand, ML is a specific subset of AI that allows systems to learn from data and improve over time without being explicitly programmed for every task. ML uses algorithms to identify patterns in large datasets and make predictions or decisions based on new data. For instance, in supervised learning, a model is trained on labeled data to predict specific outcomes, while unsupervised learning deals with unlabeled data to find hidden patterns, and reinforcement learning allows models to learn through trial and error [7]. Essentially, ML is the engine that drives many of today's AI applications, from recommendation systems and image recognition to self-driving cars and personalized healthcare. AI provides the overarching goal of creating intelligent machines, while ML offers the practical means of achieving this by developing models that can learn, adapt, and evolve, making them indispensable tools in advancing automation and problem-solving capabilities in various industries.

DL is a subfield of ML that focuses on algorithms inspired by the structure and function

of the brain. While ML includes a broad range of techniques that rely on human-engineered features for tasks such as classification and regression, DL differentiates itself by using neural networks with many layers to automatically learn these features from raw data. In DL particularly through Convolutional Neural Networks (CNNs), the field has revolutionized computer vision tasks such as image classification, object detection, and facial recognition. CNNs are a type of deep neural network that excels at extracting meaningful patterns from image data through convolutional layers, pooling layers, and fully connected layers. These networks are highly effective for tasks involving images, such as detecting whether an eye is open or closed, because they automatically learn to identify important features like edges, textures, and shapes. The key difference is that while traditional ML algorithms often require manual feature selection, DL models, especially CNNs, are capable of learning complex representations from the data with minimal human intervention [8].

An example of a DL application is eye detection, which involves detecting whether a person's eyes are open or closed using facial images. The eye detection problem differs from general object classification tasks due to differences in race, age, or gender. Furthermore, the need for diverse datasets must be addressed when deploying eye recognition systems to ensure that the model generalizes well across different populations and environments.

Transfer learning, which utilizes pre-trained CNNs, allows models trained on large datasets to be adapted for new tasks with less data and computing power. Instead of training from scratch, a pre-trained model's early layers, which capture basic features like edges and textures, are reused, while the final layers are fine-tuned for the specific task [9]. This approach is highly effective for tasks like object detection or eye state monitoring, improving performance and reducing training time, especially when only a small dataset is available.

MobileNet is an example for Pretrained CNNs. It is a family of lightweight CNN developed by Google, specifically designed for mobile and edge devices where computational resources are limited. The primary innovation behind MobileNet is its use of depth wise separable convolutions, which significantly reduce the number of parameters and computations required compared to traditional convolutional layers. In a standard convolutional layer, each filter operates across all input channels, leading to a high computational cost. However, MobileNet breaks this process into two separate layers: a depth wise convolution that applies a single filter per input channel and a pointwise convolution that combines the outputs from the depth wise layer using 1x1 convolutions. This architecture allows MobileNet to maintain competitive accuracy while being much

more efficient in terms of speed and memory usage [10].

Subsequent versions, such as MobileNetV2 and MobileNetV3, built upon this foundation by introducing additional optimizations like linear bottlenecks and squeeze-and-excitation blocks, further enhancing efficiency and accuracy. These advancements make MobileNets particularly suitable for applications in computer vision tasks such as object detection, image segmentation, and facial recognition on mobile platforms. as shown in (figure 2-1).

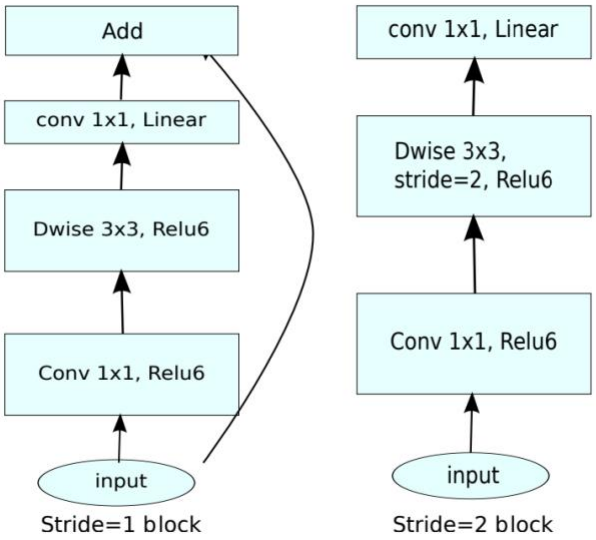


Figure 2-1 *Block Diagram of MobileNETV2*

2.1.2 Edge Computing

Edge computing is a distributed computing paradigm that brings data processing and analytics closer to where the data is generated, such as sensors, Internet of Things (IoT) devices, or cameras. Unlike traditional cloud computing, where data is sent to centralized servers for processing, edge computing enables devices at the "edge" of the network to handle computation locally. This decentralized approach significantly reduces latency, allowing for real-time data analysis and decision-making, which is crucial for applications that require immediate responses, like autonomous systems or industrial automation. By processing data locally, edge computing also conserves network bandwidth, as only necessary or summarized data is sent to the cloud, reducing overall traffic. Additionally, it enhances security and privacy, since sensitive data can be processed without leaving the local environment, minimizing the risk of exposure during transmission. Furthermore,

edge computing ensures systems can function in remote or unstable network conditions, offering greater reliability. We implement edge computing through the Arduino Nicla Vision Camera, which plays a central role in detecting driver drowsiness. The camera continuously captures images of the driver's face and using ML models embedded directly on the device, it processes this data locally to determine whether the driver's eyes are open or closed. This local processing allows the system to make decisions instantly without needing to communicate with a remote cloud server, which would introduce delays. The low latency achieved through edge computing is critical in this application, as it ensures that any signs of drowsiness are detected and acted upon quickly, potentially preventing accidents. Additionally, by processing data at the edge, we reduce the volume of information that needs to be transmitted over the network, which conserves bandwidth and allows the system to run efficiently even in environments with limited or intermittent internet connectivity. This edge-based approach ensures that the driver drowsiness detection system remains fast, reliable, and efficient, with added benefits of privacy [11].

2.1.2.1. Arduino Nicla Vision Camera

The Arduino Nicla Vision is a compact, standalone camera designed for analyzing and processing images at the edge. as shown in (figure 2-3), It is equipped with a 2 MP color camera, which allows it to capture high-quality images suitable to detect driver drowsiness by monitoring eye movement, enhancing both the speed and reliability of the system [13].

Figure 2-3 *Nicla Vision Camera*

2.2 Related Work Survey

2.2.1. An Intelligent System for Preventing Accidents Due to Driver distraction

The research focused on developing an intelligent accident prevention system designed to monitor and alert drivers in real-time when they became distracted. It integrated DL models for object and drowsiness detection, as well as head pose estimation to track the driver's focus. The system detected common causes of accidents, such as driver drowsiness and phone usage while driving. Upon identifying these distractions, it generated an audible alert to refocus the driver's attention on the road, thus enhancing road safety by providing a comprehensive, real-time monitoring solution to prevent accidents. The system included a camera for capturing real-time visual data of the driver, an Arduino Uno microcontroller for processing data from the sensors, an HC-05 Bluetooth module for transmitting data to the alarm, and a buzzer that generated an alert when distractions were detected [14].

Strengths:

The system employs a multimodal approach, utilizing multiple DL models and head pose estimation to effectively detect distractions such as drowsiness and phone use. It also includes hardware components like a buzzer, offering real-time alerts.

Weaknesses:

Potential false positives could occur, where the system might incorrectly identify distractions.

Similarity with this project:

Both systems aim to detect driver drowsiness and provide real-time alerts to enhance road safety.

Difference:

The system uses a regular camera instead of Nicla Vision, which detects head position, movement, and phone usage. Nicla Vision is better because it's designed for DL, combining a 2MP camera and motion sensor for accurate detection. It processes data on the device for faster responses and has a compact design, making it more suitable for vehicles than standard cameras.

2.2.2. Real-Time Driver Drowsiness Detection using Computer Vision

This proposed system is designed to significantly reduce the frequency of accidents caused by driver drowsiness and fatigue. using a Raspberry Pi, a camera for face monitoring, and peripheral hardware. The system employs DL models like deep CNNs for facial landmark detection, focusing on features such as eyes and mouth. It also uses shape prediction for detecting eye closure and yawning, along with Haar-cascade classifiers for tracking movements. The models are trained on the iBUG 300-W dataset for improved accuracy in detecting key facial landmarks [15].

Strengths:

Extensive data analysis utilizes large datasets from diverse conditions, including varying lighting and head positions, to ensure robust model training and validation.

Weaknesses:

The camera doesn't adjust automatically, making it less flexible. And can give confusing alerts if a driver closes their eyes and yawns at the same time.

Similarity with this project:

include a core objective of detecting drowsiness to prevent accidents and provide real-time alerts, both projects leverage AI and image processing to analyze eye patterns for drowsiness, emphasis on safety and real-time feedback with audio alerts to notify drivers, and involvement of hardware components like cameras to detect signs of drowsiness.

Differences:

Our project utilizes an Nicla Vision camera for cost-effective vehicle integration, whereas similar systems may use hardware like Raspberry Pi, affecting processing power. Our project features a lightweight AI model optimized for edge computing, while similar systems might employ more complex algorithms that require cloud processing. In terms of detection, our project focuses specifically on detecting eye closure as an indicator of drowsiness, providing real-time alerts through a sound notification.

The similar project, in addition to detecting eye closure, includes yawning detection and movement tracking, offering a broader scope of monitoring. However, this added functionality increases system complexity.

2.2.3. Evaluation of driver fatigue on two channels of EEG data

Electroencephalogram (EGG) system Determine the Optimal Indicator of Driver Fatigue: The research aims to identify which EEG-based ratio indices are the most effective in indicating driver fatigue based on DL, EEG signal is one of the most predicative and reliable indicator because it directly measures brain activity

The hardware components include Logitech camera, EGG Headset, computer or data processing unit, Electrodes (small sensors), Steering Wheel, Logitech MOMO Force Feedback Racing Wheel [16].

EEG Signal Measurement:

- EEG involves placing electrodes on the scalp to record electrical activity in the brain. Different frequency bands in EEG (e.g., alpha, beta, theta) are associated with different states of consciousness and cognitive function.
- As fatigue increases, specific changes in these frequency bands can be detected.
- The EEG data is processed alongside video and driving inputs to determine optimal fatigue indicators, which help in real-time monitoring and assessment of the driver's alertness level. It measures the biological signs in an EEG-based driver fatigue detection system:
- The EEG headset is worn by the driver and is used to collect brainwave data.
- The camera, such as the Logitech camera mentioned, is connected to a computer or processing unit to track facial expressions, eye movement, and other visual cues.
- Both the EEG headset and camera send their data to a central processing unit where the data is analyzed to determine if the driver is fatigued.

Strengths:

offer an effective way to identify when a driver is no longer alert a system that uses a camera to track the driver’s face and eyes, while simultaneously using EGG.

Weaknesses:

Complexity and Cost EEG systems are relatively complex and expensive to implement.

Similarity with this project:

this project's focus on real-time video monitoring is in line with the use of a driving simulator equipped with a Logitech camera to record facial expressions for the purpose

of detecting drowsiness.

Differences:

Unlike our project, it focuses on visual signals, this approach integrates EEG data—tracking brainwaves to assess fatigue. Although it is not directly applicable, this shows a method of combining physiological signals with visual detection for more accuracy.

2.2.4. Driver Drowsiness Detection

The research paper titled "Driver Drowsiness Detection" presents a real-time system that uses computer vision techniques to monitor and analyze a driver's facial features to detect signs of drowsiness. A camera mounted in front of the driver captures facial images, focusing primarily on the eyes. These images are processed using Python, OpenCV, and dlib libraries to track key indicators such as eye closure duration, yawning frequency, and head nodding. When the system detects that these drowsiness indicators exceed a specified threshold, it triggers an alarm to alert the driver. The system aims to reduce accidents caused by driver fatigue and drowsy driving. It is designed with the potential for integration into vehicles, offering a solution to enhance road safety by providing timely alerts to prevent accidents [17].

Strengths:

The system captures and processes driver facial images in real-time, ensuring quick detection of drowsiness symptoms like eye closure, yawning, and head nodding. In addition, The use of Python, OpenCV, and dlib makes the system easy to integrate into existing vehicle systems. It generates audible or visual alerts to warn drivers when drowsiness is detected, improving safety on the road. The system relies on widely available and reliable open-source libraries such as OpenCV, which makes it accessible and cost-effective.

Weaknesses:

The system Depending on Lighting and environment Conditions so the accuracy may degrade in low-light or extreme weather conditions, limiting its effectiveness. The system's dependence on eye closure, yawning, and head nodding may not cover all drowsiness scenarios and could result in false negatives or positives indicators. The

system requires a high-quality camera and embedded hardware, which could increase costs and complexity during installation. The system doesn't integrate additional physiological signals like heart rate, which could provide a more comprehensive detection of drowsiness.

Similarities with this Project:

The Core Objective for Both systems aim to enhance road safety by detecting drowsiness and alerting drivers in real time. Both systems use cameras to capture facial images for detecting eye closure as key indicators of drowsiness. In addition, Both projects leverage AI techniques to analyze visual cues (eye features) and provide real-time feedback. To illustrate, Like this project, the research paper system provides immediate audio alerts to the driver when drowsiness is detected.

Differences from this Project:

While this project uses an Arduino microcontroller and Nicla Vision camera, the research system uses Python with OpenCV and dlib. This project focuses more on embedded systems and edge computing, making it more self-sufficient without requiring a continuous internet connection. while the paper system is designed to detect a broader range of facial movements (e.g., eye closure , yawning), this system focuses more on simpler detection methods such as eye closure.

2.3 Proposed & Similar Systems Comparison

	This Project (Drowsiness detection System using AI and Nicla vision)	**Driver Drowsiness Detection Research Paper**	**Real-Time Driver Drowsiness Detection using Computer Vision**	**An Intelligent System for Preventing Accidents Due to Driver distraction**	**Evaluation of driver fatigue on two channels of EEG data**
Problem solved	Detecting driver drowsiness in real-time to prevent accidents caused by driver fatigue. The system aims to reduce road accidents by providing immediate alerts when the driver shows signs of drowsiness, helping prevent sleep-related accidents.	Detecting driver drowsiness in real-time to prevent accidents caused by driver fatigue.	The system addresses the issue of accidents caused by driver drowsiness and fatigue by employing advanced facial monitoring technology to detect signs of drowsiness through eye closure and yawning detection.	The system addresses driver distractions, which is a significant lead to accidents. It monitors the driver drowsiness and sends alerts to help prevent accidents caused by drowsiness or phone use.	the research aims to identify which EEG-based ratio indices are the most effective in indicating driver fatigue based on deep learning and reduce the frequency of accidents.

Domain & Users	The system is designed for use in vehicles to help prevent accidents caused by drowsy driving. It is suitable for individual drivers, operators, and vehicle manufacture rs. The system can be integrated into cars, trucks, or other vehicles to monitor driver alertness and provide timely warnings, especially during long trips.	Automotive safety, drivers, vehicle manufacturers, transportation industries.	This system targets the automotive domain, specifically focusing on driver safety and accident prevention. The primary users are drivers who may experience fatigue while driving.	The system is designed for road safety and is intended for drivers who need an intelligent, solution to monitor and alert them When getting Distracted Or drowsy while driving.	It offers an effective way to identify when a driver is no longer alert A system that uses a camera to track the driver's face and eyes, while simultaneously using EGG.

Similarities		Both aim to enhance road safety by detecting drowsiness and providing real-time feedback to alert drivers. Both use cameras to capture driver facial images, focusing on eye closure as a key indicator of drowsiness.	Both systems aim to enhance road safety by tackling driver drowsiness and fatigue, utilizing a camera to monitor the driver and focusing on visual signs of fatigue. Additionally, both target the same user, drivers who must remain alert while operating a vehicle.	Our project focus on detecting drowsiness, Both systems aim to detect driver drowsiness and provide real-time alerts to enhance road safety.	our project's focus on real-time video monitoring is in line with the use of a driving simulator equipped with a Logitech camera to record facial expressions for the purpose of detecting drowsiness.

Differences		our project uses an Arduino and Nicla Vision, focusing on embedded systems and edge computing for standalone operation without internet. The research paper relies on computer vision libraries (Python, OpenCV, dlib), which may require more processing power and lighting conditions. this project aims to detect eye Clouser.	The proposed system employs advanced deep learning techniques for facial landmark detection, monitoring multiple features like the eyes and mouth, while our project focuses solely on eye closure. Its complexity is greater due to the use of a Raspberry Pi and deep learning models, requiring significant computational resources and a training dataset for accuracy, whereas our project is simpler and more cost-	The other project uses a regular camera to check for head movement and phone use, which can be helpful but might not be very accurate. This project, however, uses Nicla Vision, which is a better choice for cars. Nicla Vision is smaller and more efficient, It has a built-in 2MP camera and a motion sensor that help it capture clear images and detect drowsiness more accurately. This means the system can quickly spot when the driver is distracted, leading to faster	Unlike our project, it focuses on visual signals, this approach integrates EEG data—tracking brainwaves to assess fatigue. Although not directly applicable, this shows a method of combining physiological signals with visual detection for more accuracy.

			effective with using Nicla Vision camera . Additionally, the proposed system may utilize various detection and alert mechanisms, while our project specifically triggers an audio alert after a defined period of eye closure.	alerts.	

Table 2-1 *Proposed & Similar systems Comparison*

Chapter 3: System Analysis

3.1. Requirements specification

3.1.1. Survey Investigation of required components

3.2. Requirements Analysis

3.2.1. Use case diagram

3.2.2. Software requirements

3.2.2.1. Arduino IDE

3.2.2.2. Edge Impulse

3.2.2.3. Visual Studio Code (VS Code)

3.2.3. Hardware Requirements

3.2.3.1. Nicla Vision

3.2.4. Functional and Non-functional Requirements

3. System Analysis

3.1 Requirement Specifications

The goal of this project is to create a prototype that will help in reducing the number of accidents caused by drowsy driving. This prototype aims to use DL to detect signs of driver fatigue and alert drivers when drowsiness is detected. We conducted an in-depth review of current technologies to select the optimal hardware and software that would meet our requirements and deliver reliable performance. Although various solutions exist for the proposed problem, we carefully examined available options and selected the Arduino Nicla Vision Camera as the primary hardware component. The software components include the Arduino IDE and Edge Impulse. To accurately specify the requirements for this project, we conducted a survey distributed to Princess Nora bint Abdul Rahman University (PNU) students who are active drivers with valid licenses. This section presents the survey details, and an analysis of the responses gathered.

3.1.1 Survey

We conducted a survey consisting of 10 questions. It was disseminated to PNU students who are active drivers with valid licenses, to better understand their needs. This survey helped us gain insights into the hardware and software components that are most likely to provide the best results for our future users. In Appendix A, we have listed all the survey questions that were presented to the participants. This survey was designed to gather insights into driving habits and perceptions of driver drowsiness detection systems among participants. The questions focused on several key areas, starting with the frequency of long-duration driving without breaks and experiences of drowsiness at the wheel. Participants were asked about their responses to feeling drowsy, such as whether they continued driving or took preventive measures like resting or consuming caffeine. Additionally, questions explored participants' interest in a drowsiness detection system and preferred alert methods, with options for subtle to standard alerts. Frequency preferences for system monitoring, the importance of operation in varied lighting conditions, and potential trust issues associated with false alarms were also assessed. Finally, participants indicated their likelihood of using a drowsiness detection system and rated its perceived importance for enhancing road safety. This survey serves as a foundation for understanding user needs and expectations in developing an effective and reliable driver alert system. The responses to questions included in the survey are shown in

the following figure, 3-1.

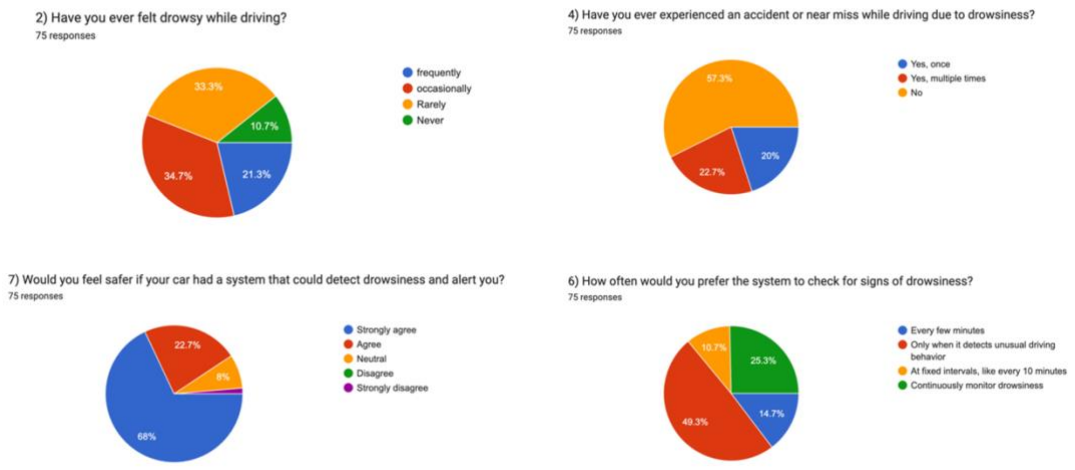


Figure 3-1 *Survey analysis*

The survey results give us useful information about drivers' experiences with drowsiness and their opinions on a drowsiness detection system. Drowsiness while driving is fairly common, with 34.7% of people saying they feel drowsy occasionally and 21.3% feeling it frequently. This means more than half of the participants have experienced drowsiness while driving at some point. Additionally, 22.7% have had an accident or close call because of drowsiness, and 20% have experienced this multiple times, showing the risks of drowsy driving. For safety, 68% of drivers strongly agree they would feel safer with a drowsiness detection system. This shows that most drivers are open to having this kind of system in their car. When asked how often the system should check for drowsiness, 49.3% prefer it to activate only when unusual driving behavior is detected, while 25.3% would like continuous monitoring. From these responses, we noticed that many drivers are concerned about drowsiness, especially on long drives, and they want the system to be accurate to avoid false alarms. To address this, we included a DL model and a Nicla Vision camera to improve accuracy in detecting drowsiness signs. Drivers also preferred an alert system that immediately alert the driver when detecting signs of fatigue, like eye closure. So, we programmed an alert system using Nicla Vision and the Arduino IDE to provide instant audio warnings when drowsiness is detected.

Most respondents said they want the alert to trigger as soon as any signs of drowsiness appear. Based on this, we programmed our system to detect drowsiness in real time using Nicla Vision, providing immediate and reliable alerts to help keep drivers safe on the road.

3.2 Requirement Analysis

3.2.1 Use case

Figure 3-2 *Use case Diagram*

The use case diagram outlines the operational flow of the driver drowsiness detection system. Initially, the system continuously monitors the driver to identify any signs of drowsiness. Upon detecting such signs, the system then classifies the driver's state as either drowsy or not. If the system determines that the driver is drowsy, it generates an alert to effectively notify the driver of their condition. This alert is designed to capture the driver's attention promptly. Ultimately, the driver is expected to respond to this alert, allowing them to take necessary actions to ensure their safety and maintain focus while driving. This process highlights the system's critical role in enhancing road safety by addressing the risks associated with driver drowsiness. as shown in (figure 3-2).

3.2.2 Software Requirements

3.2.2.1. Arduino IDE

The Arduino Integrated Development Environment (IDE) shown in (figure 3-3). is a software platform used to write, compile, and upload code to Arduino-compatible hardware, such as the Arduino microcontroller used in this project.

Figure 3-3 *Arduino IDE Interface*

A key feature of the Arduino IDE is its ability to connect hardware components like the Nicla Vision camera to the software environment [18], allowing the system to capture and process images of the driver's face to detect signs of drowsiness in real time. The IDE offers a simple, easy-to-use text editor with useful features like a message area for error reporting, a console for displaying feedback, and a toolbar for actions like saving, verifying, and uploading code to the hardware shown in (figure 3-4). In the IDE programs are called "sketches" and are written in the editor, saved with a .ino extension. The editor supports basic functions such as copy-paste, search and replace, and syntax highlighting, making it easier to manage code for this project. Once the code is ready, the toolbar allows quick uploading to the Arduino microcontroller, which handles the system's components and processes sensor data. The message area displays feedback, such as successful uploads or error messages, while the console provides more detailed logs for debugging. The lower-right corner of the IDE shows the configured board and serial port shown in (figure 3-3). ensuring the correct hardware is connected for uploading code [19].

Figure 3-4 Arduino IDE Arduino IDE text editor

To begin working with Arduino IDE:

1- Go to the Arduino website and download the latest version of the IDE for your operating system (Windows, Mac, or Linux), as shown in (figure 3-5) [20].

Figure 3-5 Arduino IDE Download

2- Open the downloaded file and follow the on-screen instructions to install the software, as shown in (figure 3-6).

3- After installation, launch the Arduino IDE, and you'll see the text editor where you can start writing your sketches.

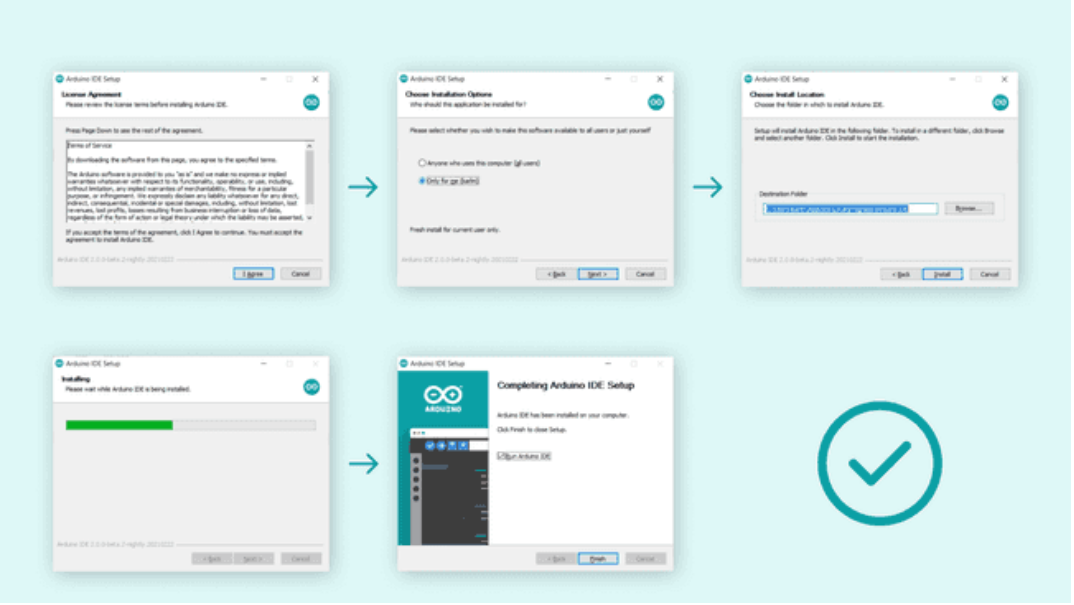


Figure 3-6 Arduino IDE instructions for installation and launching.

In this project, the Arduino IDE is used to write and upload code that allows the Arduino microcontroller to communicate with the Nicla Vision camera. This code will process the data from the camera to detect signs of driver drowsiness in real time.

3.2.2.2. Edge Impulse

Edge Impulse is a platform designed for developing, training, and deploying ML models optimized for edge devices with limited computing power, such as the Nicla Vision camera used in this project [21]. Edge Impulse plays a crucial role in training the TinyML model that processes real-time images from the camera to detect drowsiness [22]. This platform enables machine learning on devices like the Nicla Vision camera, which has limited computing power but needs to perform high-performance AI tasks in real-time [23]. It provides an intuitive interface for uploading data, such as images captured by the Nicla Vision camera, and facilitates training and deploying ML models directly to the Arduino microcontroller without the need for a constant internet connection.

Steps for using Edge Impulse to create and deploy the model:

1- Visit Edge Impulse and create an account, as shown in (figure 3-7).

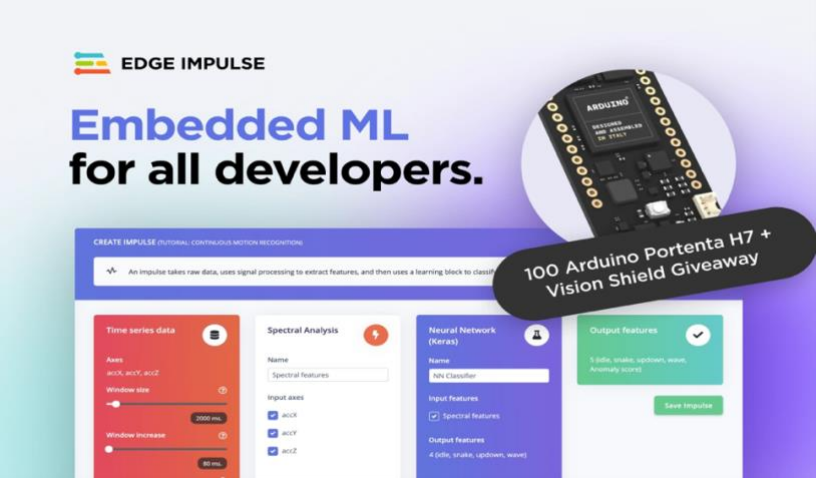


Figure 3-7 *Edge Impulse*

2- Create account, as shown in (figure 3-8).

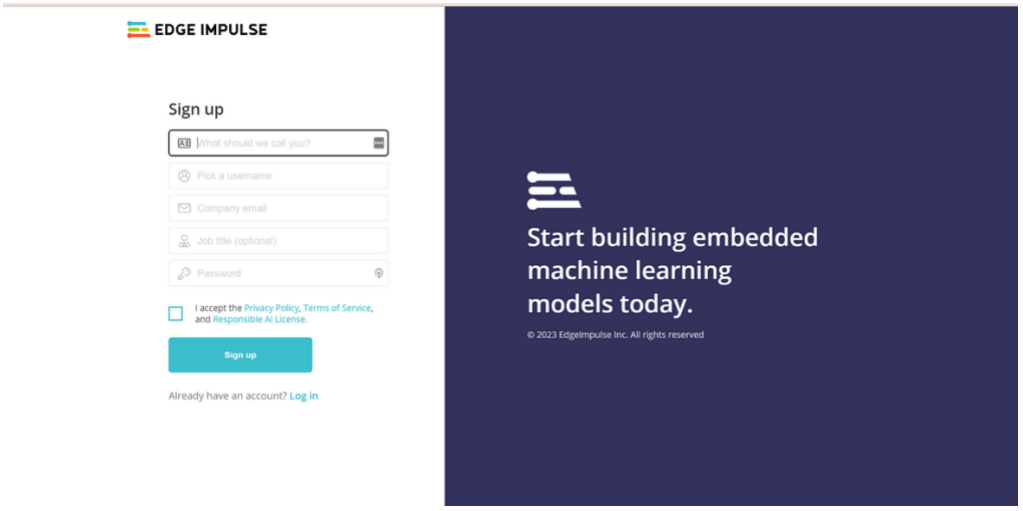


Figure 3-8 *Edge Impulse Sign-Up Page*

3- create a new project, as shown in This project will be where you upload your image data (such as images of the driver's face) and train your AI model.
4- Upload the images captured by the Nicla Vision camera, as shown in (figure 3-9). These images will be used to teach the model to recognize signs of drowsiness, eye closure.

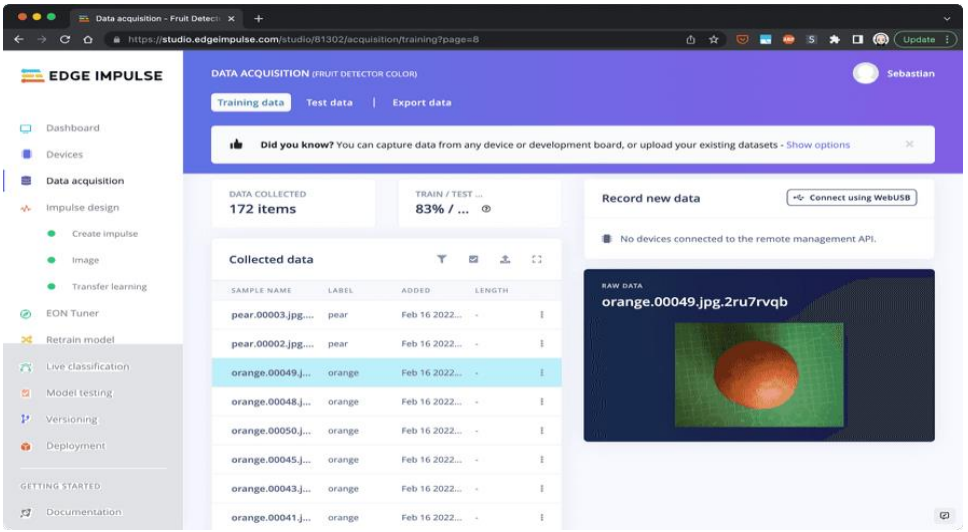


Figure 3-9 *Edge Impulse Data Acquisition Interface*

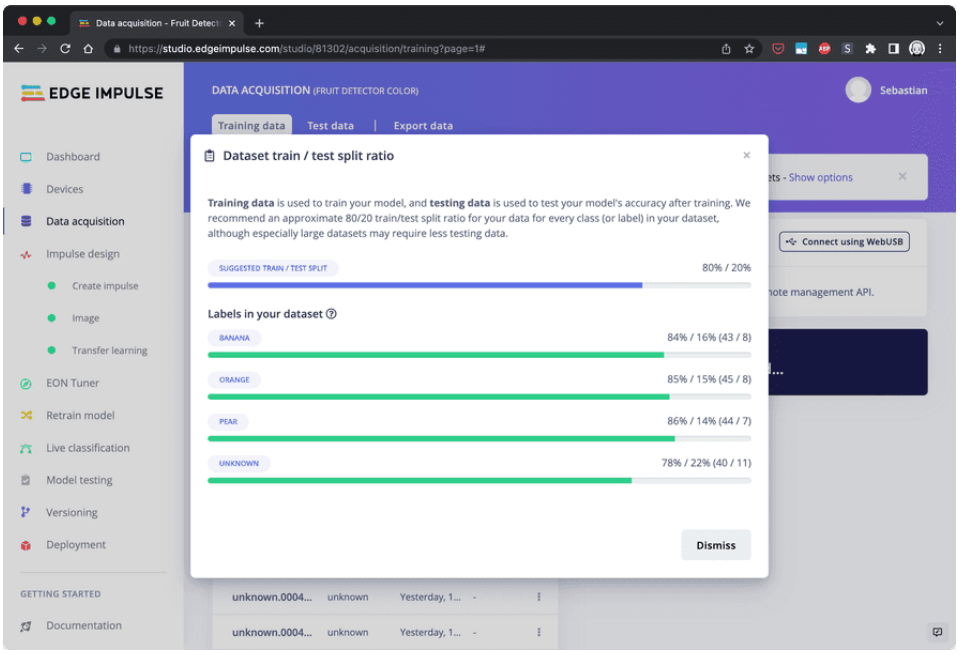


Figure 3-10 *Dataset Train/Test Split in Edge Impulse*

1- Use the tools on Edge Impulse to train the model. The platform will help you process the images and trine a ML model that can run on your Arduino , as shown in (Figure 3-11) and (Figure 3-12).

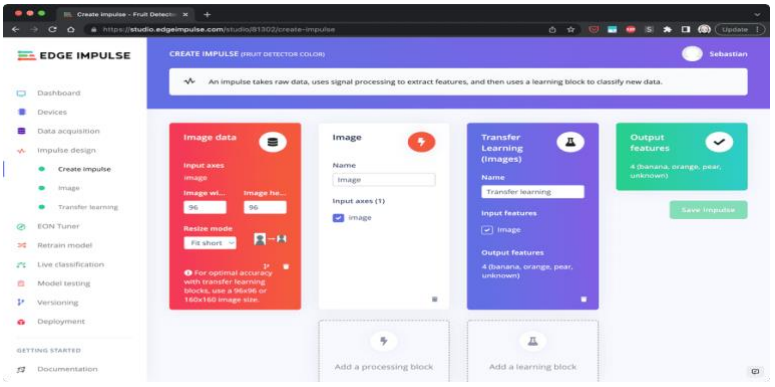


Figure 3-11 *Create Impulse Interface in Edge Impulse*

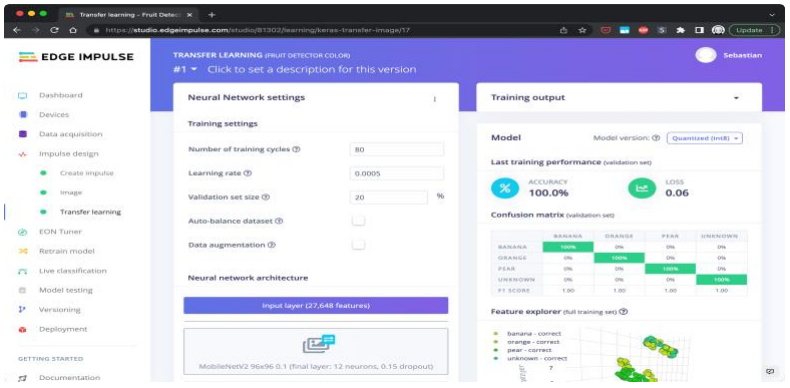


Figure 3-12 *Transfer Learning Settings in Edge Impulse*

2- Once the model is trained, export it to a format compatible with the Arduino microcontroller than upload this model to your Arduino microcontroller using the Arduino IDE, as shown in (Figure 3-13).

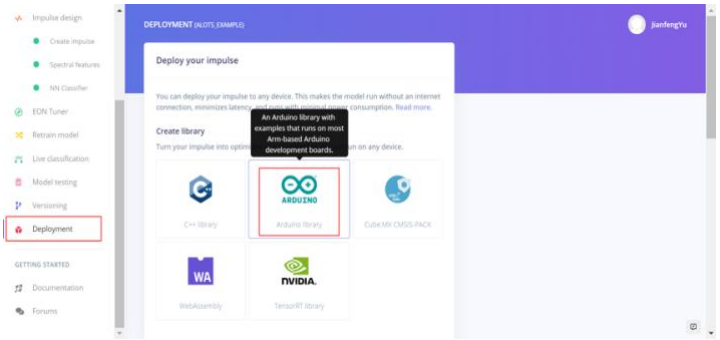


Figure 3-13 *Deployment Options in Edge Impulse*

Edge Impulse's user-friendly interface makes it straightforward to deploy AI solutions at the edge, which is essential for this project's as real-time image processing is required without dependence on cloud resources [24].

3.2.2.3. Visual Studio Code (VS Code)

Visual Studio Code (VS Code) is a free, open-source code editor developed by Microsoft that supports development in multiple programming languages. Known for its speed, flexibility, and a wide variety of extensions, VS Code has become a popular choice for software developers and data scientists alike.

VS Code's customizable interface allows users to personalize their workspace according to their preferences, including choosing color themes, keyboard shortcuts, and layout styles. As shown in Figure 3-14, the initial setup screen presents several options for customizing the editor's appearance, such as selecting between dark, light, or high contrast themes.

It features integrated terminal support, Git integration, debugging tools, and extensions for platforms such as Python, C++, Arduino, and many others as shown in figure 3-15, making it highly adaptable for different development workflows [25].

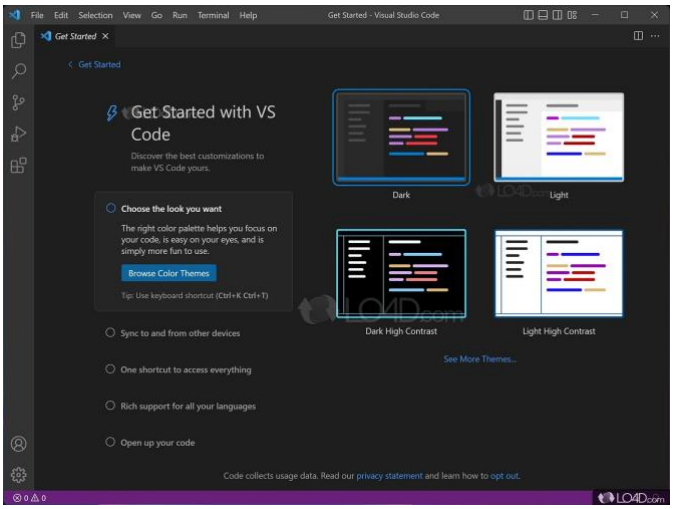


Figure 3-14 *Visual Studio Code interface*

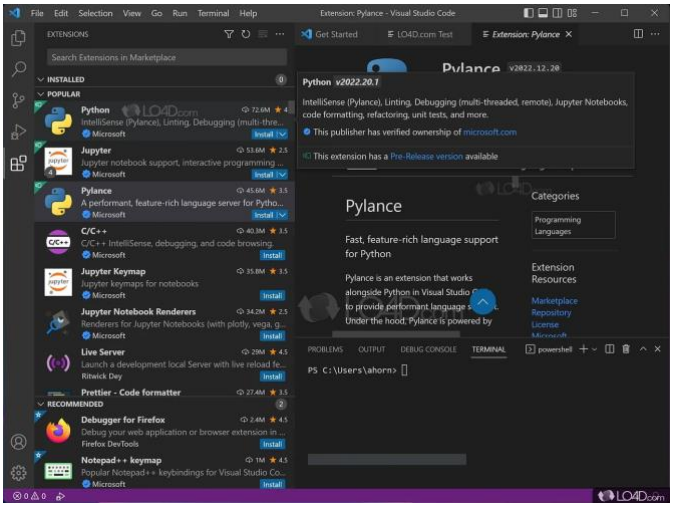


Figure 3-15 *Visual Studio Code extensions*

VS Code comes with a simple and intuitive layout that maximizes the space provided for the editor, while leaving enough space to browse and access the full context of your folder or project. The user interface is divided into five main areas:

- **Editor** – The main area to edit your files. You can open as many editors as you like, side by side, both vertically and horizontally.
- **Primary Side Bar** – Contains different views like the Explorer to assist you while working on your project.
- **Status Bar** – Displays information about the opened project and the files you edit.
- **Activity Bar** – Located on the far left-hand side. Let's you switch between views and give you additional context-specific indicators, such as the number of outgoing changes when Git is enabled. You can also change the position of the Activity Bar.
- **Panel** – An additional space for views located below the editor region. By default, it contains output, debug information, errors and warnings, and an integrated

terminal. The Panel can also be moved to the left or right for more vertical space. These elements work together to provide a seamless coding experience, adaptable to various workflows and development needs as shown in Figure 3-16.

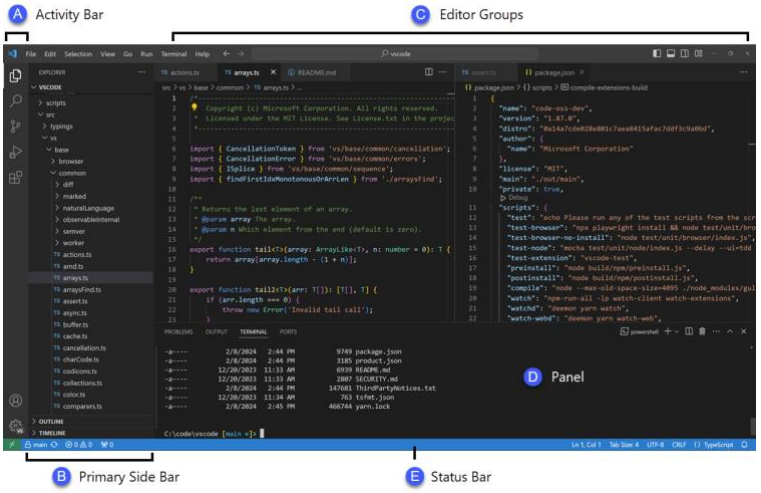


Figure 3-16 *Visual Studio Code program editor*

3.2.3 Hardware Requirements

Hardware	Figure	Brief description
Nicla Vision camera		The Nicla Vision is a compact AI camera with a 2MP sensor, motion detection, and onboard processing, ideal for real-time image recognition and edge computing applications.

Table 3-1 *Hardware Requirements*

3.2.3.1 Nicla vision camera

Its compact size and low power consumption make it ideal for various applications. These features make Nicla Vision perfect for rapid prototyping and developing edge-based image processing and machine vision solutions, including asset tracking, object recognition, predictive maintenance, and more [13]. as shown in (figure 3-17).

Figure 3-17 *Nicla Vision*

3.2.4 Functional and Non-functional Requirements

Functional requirements:

- System should continuously monitor the driver's eye closure.
- System should process the data from the camera and sensors in real-time using the Arduino microcontroller.
- System should be capable of Detecting the driver's eyes, If the eyes remain closed for more than 3 seconds, it will classify the image as indicating a drowsy driver.
- System should trigger an alert (sound) when drowsiness is detected.

Non-functional requirements:

- Accuracy: the system should detect drowsiness with a high level of accuracy.
- Response time: the system should be done in a certain period in detecting motion and sending an alert.

Chapter 4: System Design

4.1. System Design

4.1.1. System architecture

4.1.2. System Prototype and Activity Diagram

4.1.3. System Flowchart

4.1.4. User interface design

4. System Design

4.1 System Design

The Driver Drowsiness Detection System is designed as a prototype to improve road safety by monitoring drivers for signs of drowsiness and alerting them when needed. The system includes two main parts: a Nicla Vision camera and an alert system. These parts work together to detect and respond to drowsy behavior. The design is simple and modular, meaning each part has a specific job. The Nicla Vision camera captures real-time images of the driver's eyes, the Nicla Vision is trained/tested using MobileNet DL model to decide if the driver is drowsy, the classification results are then used to trigger an alert through sound if needed. By combining these parts, the system ensures reliable monitoring and timely alerts to help drivers stay safe on the road.

4.1.1 System architecture

system architecture of this project is made up of a Nicla Vision camera, which captures real-time images of the driver's eyes to monitor eye closure duration. The camera runs AI algorithms that analyze the images in real time to determine if the driver is drowsy [26]. If the driver's eyes remain closed for more than three seconds, the system triggers an alert through a buzzer to get the driver's attention. If no signs of drowsiness are detected, the alert system remains inactive. This ensures accurate and timely alerts to help improve road safety, as shown in (figure 4-1).

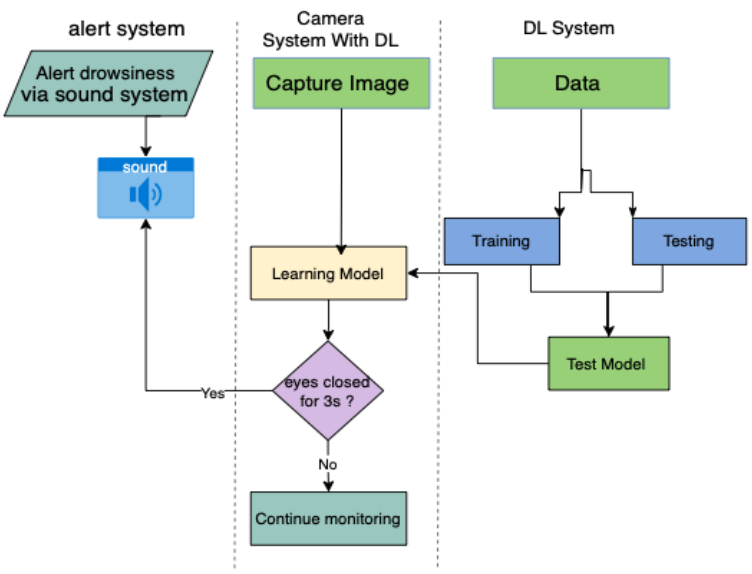


Figure 4-1 *system architecture*

4.1.2. System Prototype and Activity Diagram

The system is composed of three primary modules: Detection, recognition and Alert Activation. Each module works together to ensure the driver's alertness is constantly monitored and appropriate measures are taken if signs of drowsiness are detected as shown in (figure 4-2). And the activity diagram is depicted in (figure 4-3).

First Module Eyes Closure Detection, the purpose of this module is to analyze the driver's facial features and detect signs of drowsiness in real-time. Using the Nicla Vision camera, this module continuously captures and processes visual data to identify potential drowsiness indicators, such as eye closure. This data is processed by an AI model trained using MobeilNet in Edge Impulse, which interprets the driver's behavior and determines the drowsiness level. However, the captured image should be conformed that the driver is drowsy by the second module. Eye State Recognition Module, which will checked if the symptoms of drowsiness like eye closure is detected. The final module, will only be activated if drowsiness conformed, will then an alert sound be immediately triggered within the vehicle to notify the driver, ensuring they are aware of their drowsy state.

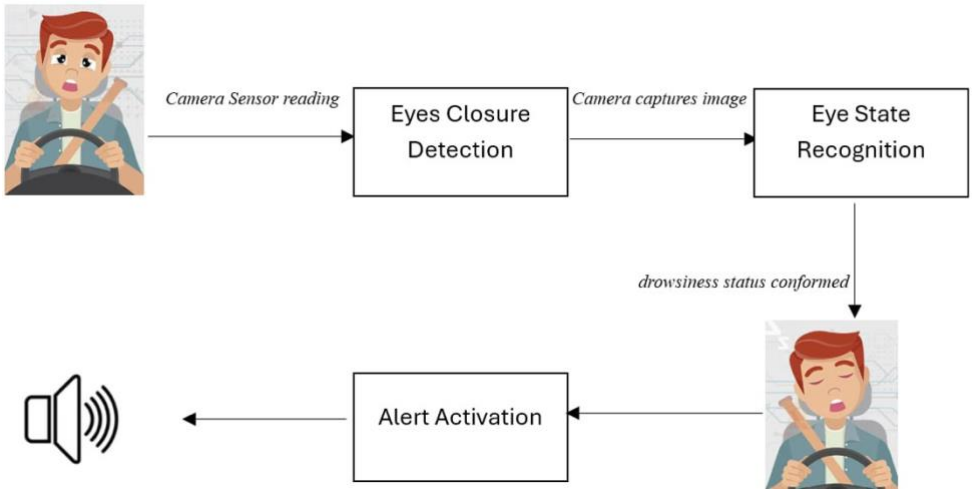


Figure 4-2 *System Prototype Modules*

Activity Diagram

The flow of the activities as shown in (figure 4-2). includes the following:

1. **Monitor Driver's Eyes:**

   Continuously track the driver's eyes to check if they are open or closed.

2. **If Closed Monitor Driver's Eyes:**

   Start counting how long the eyes stay closed.

3. **If Open:**

   Reset the counter and continue monitoring.

4. **Duration Check:**

   Less than 3 seconds: If eyes are closed for less than 3 seconds, return to monitoring. 3 seconds or more: If eyes remain closed for 3 seconds or more, proceed to trigger alerts.

5. **Trigger Alerts:**

   Activate an audio alert first to warn the driver.

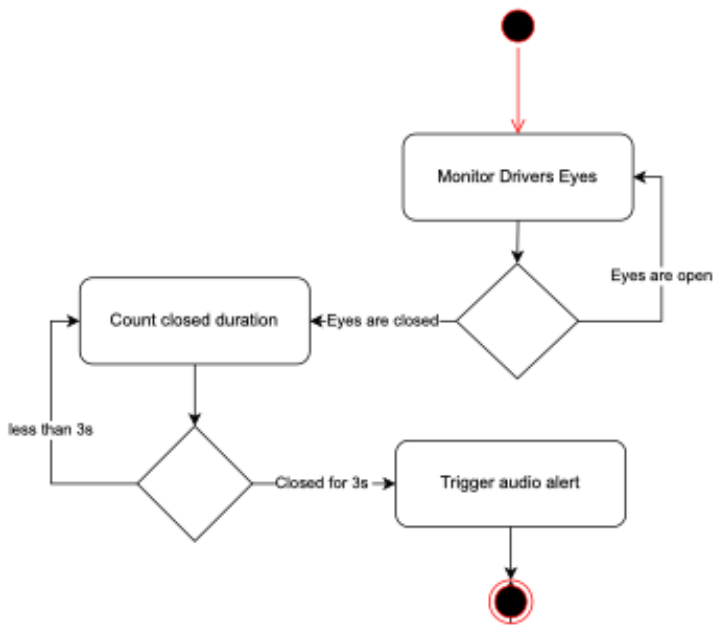


Figure 4-3 *Activity Diagram*

4.1.3. System Flowchart

Figure 4-3 illustrates the operational steps of the driver drowsiness detection system. When the process begins, the system continuously monitors the driver's eyes using the Nicla Vision camera. If the eyes are detected as closed for more than three seconds, the camera captures an image for further analysis. The system then classifies the image to determine whether the driver is drowsy or not. If drowsiness is confirmed, an alarm is generated to alert the driver. Otherwise, the system continues monitoring, ensuring the driver stays awake and attentive.

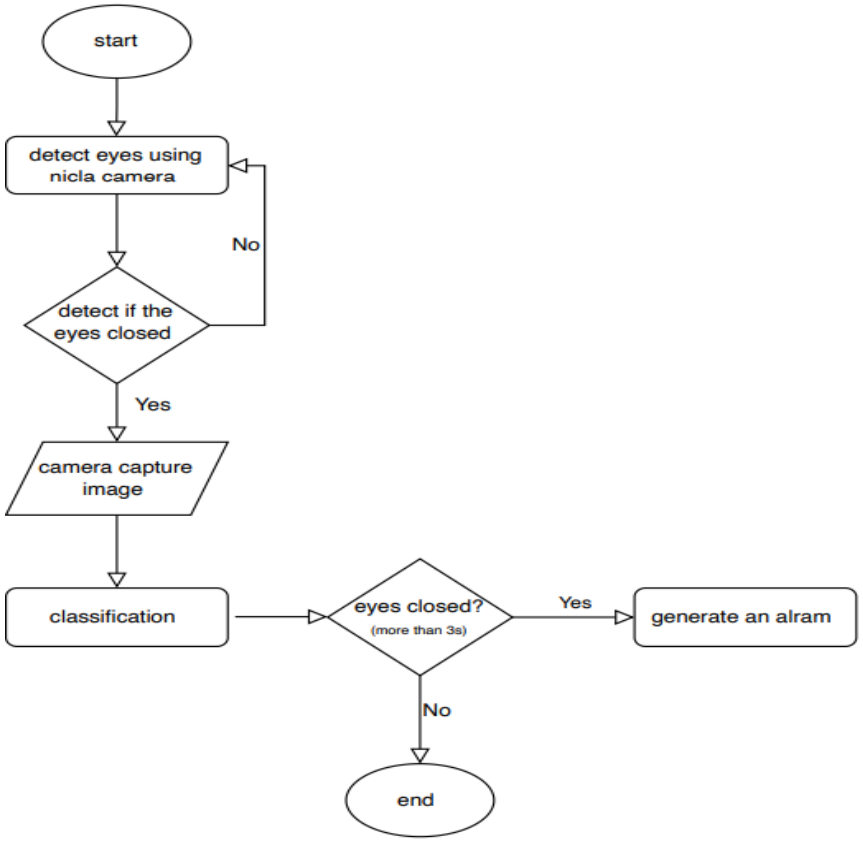


Figure 4-4 *Flowchart*

4.1.4 User interface design

Our project is a prototype designed to demonstrate the core functionality of detecting driver drowsiness through real-time eye state analysis. Since the primary focus is on delivering a hardware prototype rather than a complete application, a user interface design was considered unnecessary. The system is designed to operate autonomously, issuing alerts or warnings through audible signals without requiring any user interaction. Including a User Interface (UI) in a prototype at this stage would add unnecessary complexity and divert focus from the project's primary objective, which is to validate the technical feasibility of drowsiness detection.

Chapter 5: Implementation

5.1. Implementation Requirements

5.1.1. Hardware Requirements

5.1.2. Software Requirements

5.2. Visual Studio Code configuration

5.2.1. Model Code Configuration

5.3. Edge Impulse Configuration

5.4. Arduino IDE Configuration

5.4.1. Arduino Code

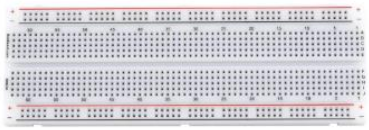
5.5. The Proposed Architecture

5. Implementation

5.1. Implementation Requirements

5.1.1 Hardware Requirements

The hardware implementation involves assembling the physical components that support data acquisition, processing, and output alerts. The primary device used is the Arduino Nicla Vision, which combines a camera and edge processing in a compact module. Additional components like jumper wires, a breadboard, and a buzzer are used to complete the system. The buzzer acts as the alert mechanism triggered by the detection algorithm. An NPN transistor is used as a switch to control the buzzer, while a resistor is added to limit the current and protect the circuit.

Component	Image
Nicla Vision Camera	
Jumper Wires	
Breadboard	
Buzzer	

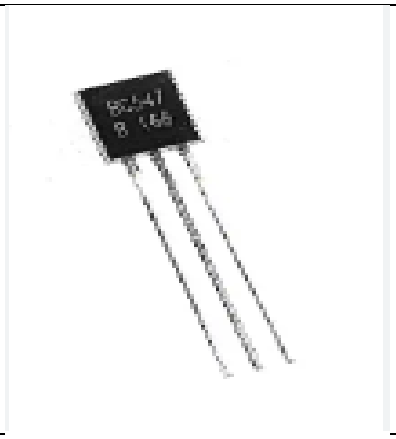
NPN Transistor	
Resistor	

***Table5-1**: Hardware Component*

5.1.2 Software Requirements

The software implementation supports the development and training of the AI model, its deployment on the Nicla Vision device, and programming logic for real-time operations. The workflow includes coding the system using Arduino IDE for hardware logic, training the machine learning model using Edge Impulse, and building and testing the AI model using Visual Studio Code is used as a code editor for various programming task.

Software	Image
Arduino IDE	
Edge Impulse	EDGE IMPULSE

Visual Studio Code	

Table5-2: *Software Requirements*

5.2. Visual Studio Code configuration

To implement the drowsiness detection model, Visual Studio Code was used as the primary development environment. This setup allowed efficient integration of Python-based machine learning frameworks and Arduino support for deploying the model to the Nicla Vision board. After configuring the necessary extensions and dependencies, the process of building, training, and evaluating the model was carried out within this environment. The following sections outline the main stages of model development, starting from data collection to performance evaluation.

1. **Data Collection**

The first step in creating an accurate AI model is collecting relevant, high-quality data. For this project, we used the Driver Drowsiness Dataset (DDD) available on Kaggle [27]. This dataset contains over 41,790 cropped face images categorized into two classes: *Drowsy* and *Non-Drowsy*. These images were originally extracted from videos using VLC media player. Dataset Properties:

- Image Type: RGB
- Number of Classes: 2 (Drowsy & Non-Drowsy)
- Image Size: 227 × 227 pixels
- Total Images: 41,790+
- File Size**: 2.32 GB

2. **Data preparation**

The preparation before training, the dataset was split into:

Training Set: 80% of the dataset to teach the AI model.

Testing Set: 20% used later to evaluate the model's performance.

Images were normalized and reshaped to meet the input shape requirements of the MobileNet model. The data was also augmented with transformations such as rotation, flipping, and zooming to increase model robustness and generalization.

3. **Model Training**

Using TensorFlow and Keras, we built a MobileNet model. MobileNet was chosen

due to its efficiency and performance on edge devices like Nicla Vision. We used transfer learning, initializing the model with pre-trained weights and fine-tuning the last layers on our custom dataset. This enabled the model to learn distinguishing features such as eye closure from drowsy faces with minimal training time and resources.

3.1 Learning Parameters

The learning parameters employed in training our MobileNet-based drowsiness detection model is listed in Table 5-3. These parameters were carefully selected to ensure stable learning behavior and good performance on edge devices such as the Nicla Vision camera.

Batch size controls how many samples are processed at once during training. We used a batch size of **32**, meaning 32 training images are passed through the network before updating the model's weights. Smaller batch sizes may help the model generalize better due to noisier gradient updates, while larger ones provide more stable updates at the cost of memory and time. A batch size of 32 provided a good balance between training efficiency and model performance.

The **initial learning rate** was set to **1e-5**, which is a very small value to ensure slow but stable convergence. The learning rate determines the size of steps the model takes in adjusting its internal weights. A high learning rate may speed up training but risks overshooting optimal values, while a low learning rate like 1e-5 allows more controlled and fine-grained updates, which is especially useful when fine-tuning pre-trained models.

The **validation frequency** was set to **1**, which means the model's performance on the validation set was evaluated after every training epoch. This helps monitor training progress closely and prevent overfitting early on.

Dropout was used as a regularization technique to reduce overfitting. We applied two dropout rates: **0.5** and **0.4**, depending on the layer. This means that during each training iteration, 40–50% of the neurons are randomly deactivated, forcing the network to learn redundant representations and improve generalization. Lastly, the **number of epochs** determines how many complete passes the model makes through the training dataset. We tested the model with multiple epoch values: **10, 20, 30, 40, 50, and 100**. Among these, the best performance was observed at **50 epochs**, where the model achieved optimal accuracy and stability without overfitting.

These learning parameters contributed significantly to the effectiveness and deploy ability of our model in real-time drowsiness detection scenarios on edge AI hardware.

Learning Parameters	Values
Batch Size	32
Initial Learning Rate	1e-5
Validation Frequency	1
Dropout	0.5 , 0.4
No. of Epochs	10, 20, 30, 40, 50, and 100

Table5-3: *Learning Parameters*

4. Model Evaluation

After training, the model's performance was evaluated using the 20% test set. Metrics such as accuracy, precision, recall, and confusion matrix were used to assess the results. A high accuracy score and balanced precision-recall values indicated the model's reliability in differentiating between drowsy and alert drivers.

Graphs and visualizations (e.g., loss vs. accuracy over epochs) were plotted using matplotlib to monitor model behavior and ensure proper learning progression.

5.2.1 Model Code Configuration

Before starting the model training, it is essential to prepare the coding environment by installing the necessary Python libraries. These libraries provide the tools needed for image processing, data handling, deep learning, visualization, and model evaluation. The environment setup ensures that all dependencies required for building and testing the AI model are available and compatible.

To build and train the driver drowsiness detection model, we first set up the programming environment in Visual Studio Code using a virtual Python environment. The following libraries illustrated in table5-4 were installed to support the complete machine learning pipeline**.**

Library	Purpose
TensorFlow	The core deep learning framework used to build and train the MobileNet model.
OpenCV-python	For image processing, reading, and manipulating the dataset images.
NumPy	Supports numerical operations and array manipulation.

matplotlib	Used to plot training accuracy/loss graphs and visualize performance.
pandas	Assists in data handling and preprocessing (e.g., reading CSV labels).
scikit-learn	Provides tools for model evaluation like accuracy, precision, and confusion matrix.

***Table 5-4**: Essential Libraries used*

The following figures show building the code model.

```python
import os
import shutil
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, Dropout, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from sklearn.model_selection import train_test_split
```

[28] Python

Configuration & Directories Setup

```python
DATA_DIR = os.path.join(os.getcwd(), "Driver Drowsiness Dataset (DDD)")
DROWSY_DIR = os.path.join(DATA_DIR, 'Drowsy')
NON_DROWSY_DIR = os.path.join(DATA_DIR, 'Non Drowsy')
OUTPUT_DIR = "prepared_data"

# Validate dataset availability
assert os.path.exists(DROWSY_DIR), "Drowsy folder not found"
assert os.path.exists(NON_DROWSY_DIR), "Non-drowsy folder not found"

# Create dataset partitions
for split in ["train", "val", "test"]:
    for category in ["drowsy", "non_drowsy"]:
        os.makedirs(os.path.join(OUTPUT_DIR, split, category), exist_ok=True)
```

[29] Python

***Figure 5-1:** Python Code for Configuration and Directory Setup.*

Data Preprocessing & Splitting

```python
def split_and_copy(source_dir, target_dirs, split_ratios):
    filenames = os.listdir(source_dir)
    train, test = train_test_split(filenames, test_size=split_ratios[1] + split_ratios[2], random_state=42)
    val, test = train_test_split(test, test_size=split_ratios[2] / (split_ratios[1] + split_ratios[2]), random_state=42)

    for split_name, files in zip(["train", "val", "test"], [train, val, test]):
        category_name = os.path.basename(source_dir).lower().replace(" ", "_")
        category_path = os.path.join(target_dirs[split_name], category_name)
        os.makedirs(category_path, exist_ok=True)
        for file in files:
            shutil.copy(os.path.join(source_dir, file), os.path.join(category_path, file))

split_and_copy(DROWSY_DIR, {"train": os.path.join(OUTPUT_DIR, "train"), "val": os.path.join(OUTPUT_DIR, "val"), "test": os.path.join(OUTPUT_DIR, "test")}, (0.7, 0.2, 0.1))
split_and_copy(NON_DROWSY_DIR, {"train": os.path.join(OUTPUT_DIR, "train"), "val": os.path.join(OUTPUT_DIR, "val"), "test": os.path.join(OUTPUT_DIR, "test")}, (0.7, 0.2, 0.1))
```

[30] Python

Data Augmentation & Generators

```python
train_datagen = ImageDataGenerator(rescale=1.0/255.0, rotation_range=20, zoom_range=0.15, width_shift_range=0.2,
                                   height_shift_range=0.2, shear_range=0.15, horizontal_flip=True, fill_mode="nearest")
val_datagen = ImageDataGenerator(rescale=1.0/255.0)

train_generator = train_datagen.flow_from_directory(os.path.join(OUTPUT_DIR, "train"), target_size=(96, 96), batch_size=32, class_mode='binary')
val_generator = val_datagen.flow_from_directory(os.path.join(OUTPUT_DIR, "val"), target_size=(96, 96), batch_size=32, class_mode='binary')
```

[31] Python

```
Found 29254 images belonging to 2 classes.
Found 8359 images belonging to 2 classes.
```

***Figure 5-2:** Data Preprocessing, Splitting, and Augmentation.*

Model Architecture

```python
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(96, 96, 3), alpha=0.35)
base_model.trainable = False

x = GlobalAveragePooling2D()(base_model.output)
x = Dropout(0.3)(x)
x = Dense(64, activation='relu')(x)
x = Dropout(0.3)(x)
output = Dense(1, activation='sigmoid')(x)

model = Model(inputs=base_model.input, outputs=output)
```

[32] Python

***Figure 5-3:** MobileNetV2-Based Model Architecture.*

Evaluation Function

```python
eval_results = []

def evaluate_model(model, test_generator, epoch):
    y_true = test_generator.classes
    y_pred_prob = model.predict(test_generator)
    y_pred = (y_pred_prob > 0.5).astype(int).flatten()

    # Confusion matrix
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()

    accuracy = (tp + tn) / (tp + tn + fp + fn)
    sensitivity = tp / (tp + fn) if (tp + fn) != 0 else 0  # Recall
    specificity = tn / (tn + fp) if (tn + fp) != 0 else 0
    precision = tp / (tp + fp) if (tp + fp) != 0 else 0
    fnr = fn / (fn + tp) if (fn + tp) != 0 else 0
    fpr = fp / (fp + tn) if (fp + tn) != 0 else 0
    auc = roc_auc_score(y_true, y_pred_prob)
    mcc = matthews_corrcoef(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)

    eval_results.append({
        "Epoch": epoch,
        "TP": tp, "TN": tn, "FP": fp, "FN": fn,
        "Accuracy": accuracy,
        "Sensitivity": sensitivity,
        "Specificity": specificity,
        "Precision": precision,
        "False Negative Rate": fnr,
        "False Positive Rate": fpr,
        "AUC": auc,
        "Matthews CorrCoef": mcc,
        "F1-Score": f1
    })

    # Plot Confusion Matrix
    plt.figure(figsize=(6, 4))
```


```python
    # Confusion matrix
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()

    accuracy = (tp + tn) / (tp + tn + fp + fn)
    sensitivity = tp / (tp + fn) if (tp + fn) != 0 else 0  # Recall
    specificity = tn / (tn + fp) if (tn + fp) != 0 else 0
    precision = tp / (tp + fp) if (tp + fp) != 0 else 0
    fnr = fn / (fn + tp) if (fn + tp) != 0 else 0
    fpr = fp / (fp + tn) if (fp + tn) != 0 else 0
    auc = roc_auc_score(y_true, y_pred_prob)
    mcc = matthews_corrcoef(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)

    eval_results.append({
        "Epoch": epoch,
        "TP": tp, "TN": tn, "FP": fp, "FN": fn,
        "Accuracy": accuracy,
        "Sensitivity": sensitivity,
        "Specificity": specificity,
        "Precision": precision,
        "False Negative Rate": fnr,
        "False Positive Rate": fpr,
        "AUC": auc,
        "Matthews CorrCoef": mcc,
        "F1-Score": f1
    })

    # Plot Confusion Matrix
    plt.figure(figsize=(6, 4))
```

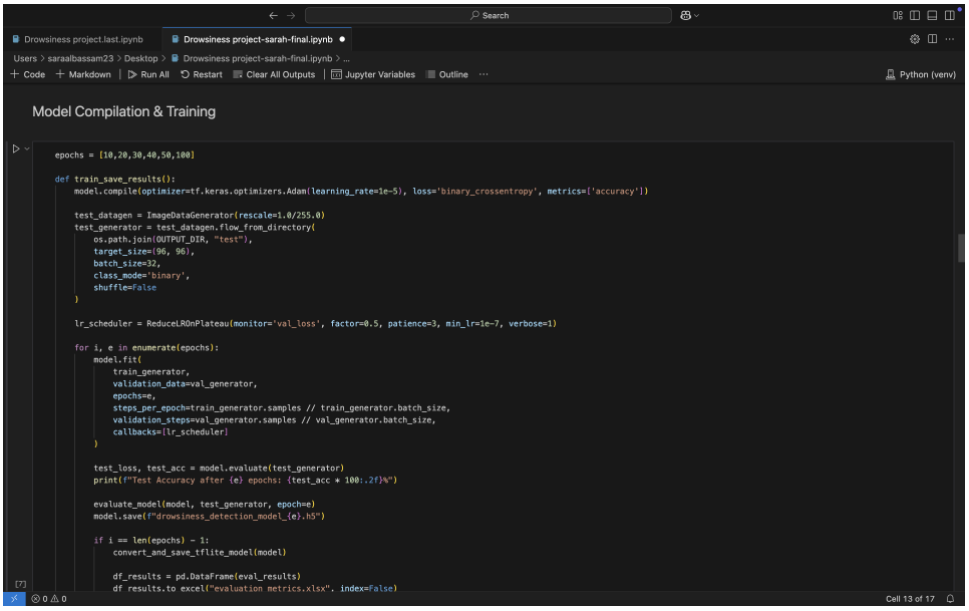

```python
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Non-Drowsy', 'Drowsy'], yticklabels=['Non-Drowsy', 'Drowsy'])
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title(f'Confusion Matrix - Epoch {epoch}')
    plt.show()

    print("Classification Report:")
    print(classification_report(y_true, y_pred, target_names=['Non-Drowsy', 'Drowsy']))
```

Figure 5-4: *Evaluation Function for Model Performance Metrics and Confusion Matrix.*

Convert Model to TensorFlow Lite (TFLite) with Quantization

```python
def convert_and_save_tflite_model(model, output_path="/Users/saraalbassam23/Desktop/gp-2-model/drowsiness_detection_model_quantized_fixed.tflite"):
    def representative_dataset():
        for _ in range(100):
            data = np.random.rand(1, 96, 96, 3).astype(np.float32)
            yield [data]

    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    converter.inference_input_type = tf.uint8
    converter.inference_output_type = tf.uint8
    converter.representative_dataset = representative_dataset

    tflite_quantized_model = converter.convert()
    with open(output_path, "wb") as f:
        f.write(tflite_quantized_model)

    print("TFLite model conversion and quantization complete.")
```

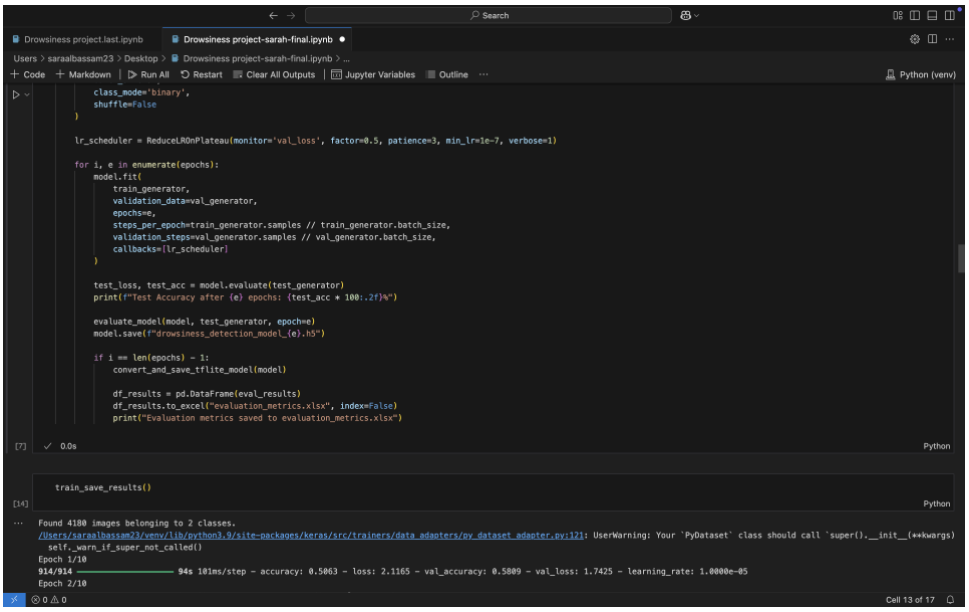
Figure 5-5: *Model Conversion to TensorFlow Lite (TFLite) with Quantization.*



Model Compilation & Training

```python
epochs = [10,20,30,40,50,100]

def train_save_results():
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), loss='binary_crossentropy', metrics=['accuracy'])

    test_datagen = ImageDataGenerator(rescale=1.0/255.0)
    test_generator = test_datagen.flow_from_directory(
        os.path.join(OUTPUT_DIR, "test"),
        target_size=(96, 96),
        batch_size=32,
        class_mode='binary',
        shuffle=False
    )

    lr_scheduler = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-7, verbose=1)

    for i, e in enumerate(epochs):
        model.fit(
            train_generator,
            validation_data=val_generator,
            epochs=e,
            steps_per_epoch=train_generator.samples // train_generator.batch_size,
            validation_steps=val_generator.samples // val_generator.batch_size,
            callbacks=[lr_scheduler]
        )

        test_loss, test_acc = model.evaluate(test_generator)
        print(f"Test Accuracy after {e} epochs: {test_acc * 100:.2f}%")

        evaluate_model(model, test_generator, epoch=e)
        model.save(f"drowsiness_detection_model_{e}.h5")

        if i == len(epochs) - 1:
            convert_and_save_tflite_model(model)

            df_results = pd.DataFrame(eval_results)
            df_results.to_excel("evaluation_metrics.xlsx", index=False)
```

Figure 5-6 *Model Compilation, Training, and Export to TensorFlow Lite.*



```python
        class_mode='binary',
        shuffle=False
    )

    lr_scheduler = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-7, verbose=1)

    for i, e in enumerate(epochs):
        model.fit(
            train_generator,
            validation_data=val_generator,
            epochs=e,
            steps_per_epoch=train_generator.samples // train_generator.batch_size,
            validation_steps=val_generator.samples // val_generator.batch_size,
            callbacks=[lr_scheduler]
        )

        test_loss, test_acc = model.evaluate(test_generator)
        print(f"Test Accuracy after {e} epochs: {test_acc * 100:.2f}%")

        evaluate_model(model, test_generator, epoch=e)
        model.save(f"drowsiness_detection_model_{e}.h5")

        if i == len(epochs) - 1:
            convert_and_save_tflite_model(model)

            df_results = pd.DataFrame(eval_results)
            df_results.to_excel("evaluation_metrics.xlsx", index=False)
            print("Evaluation metrics saved to evaluation_metrics.xlsx")
```

```python
train_save_results()
```

```
Found 4180 images belonging to 2 classes.
/Users/saraalbassam23/venv/lib/python3.9/site-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)
  self._warn_if_super_not_called()
Epoch 1/10
914/914 ━━━━━━━━━━━━━━━━━━━━ 94s 101ms/step - accuracy: 0.5063 - loss: 2.1165 - val_accuracy: 0.5809 - val_loss: 1.7425 - learning_rate: 1.0000e-05
Epoch 2/10
```

Figure 5-7 *Epoch Loop Execution and Training Results Output.*

5.3 Edge Impulse Configuration

Edge Impulse plays a critical role in this project by bridging the gap between the trained machine learning model and the hardware components used for real-time drowsiness detection. The platform enables the integration of TinyML models into edge devices such as the Arduino Nicla Vision, allowing for efficient on-device processing without cloud dependency.

In this project, Edge Impulse is used for two main purposes:

A) Deploying the AI model as an Arduino library to integrate with the Arduino IDE, enabling us to write logic to trigger the buzzer when drowsiness is detected.

B) Deploying the model directly to the Nicla Vision camera for standalone

drowsiness detection based on captured facial images.

A. Uploading and Exporting the Model as an Arduino Library

The first step is to upload the trained machine learning model to the Edge Impulse platform. As shown in figure5-8.

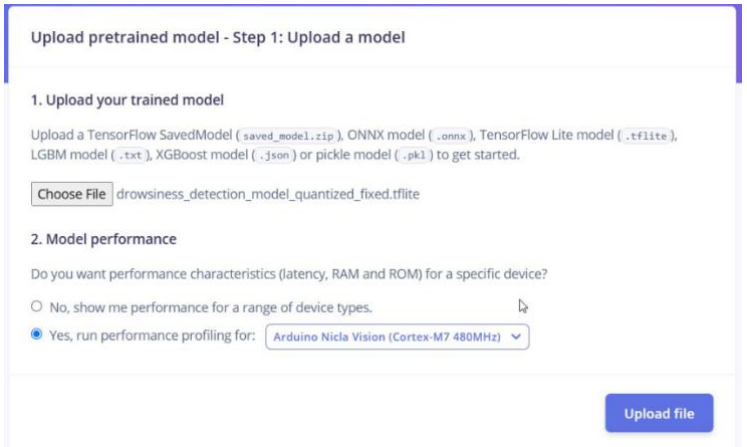


Figure 5-8: *Uploading the model.*

Once the model was uploaded and the configuration was complete, we saved the impulse settings. As shown in figure5-9.

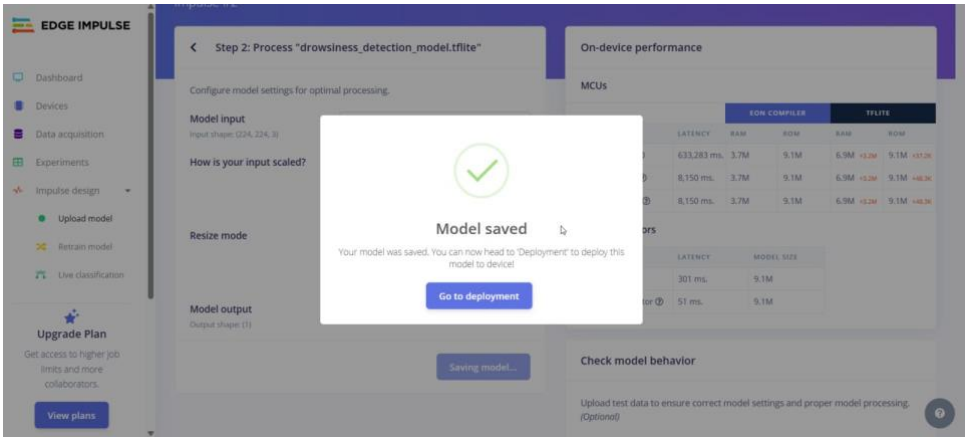


Figure 5-9 *Model saved.*

We navigated to the Deployment tab in Edge Impulse. From the Default Deployment options, we selected Arduino Library and clicked Build. Edge Impulse generated a .zip file containing our entire model in a format that will be used inside the Arduino IDE.

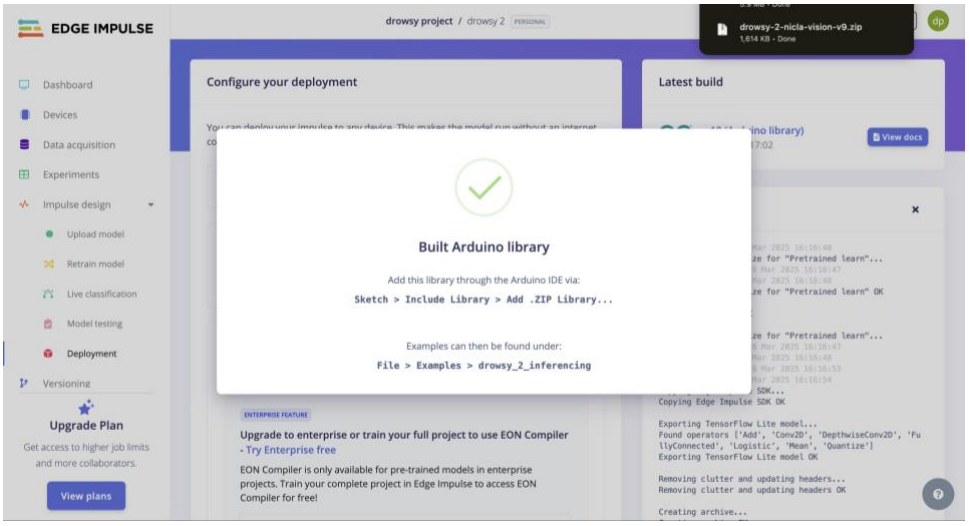


Figure 5-10: *the model .zip file.*

B. Flashing the Model to Nicla Vision Using Edge Impulse CLI

First, we install the edge impulse CLI as shown in figure5-11.

```
Last login: Mon Mar 24 22:10:06 on ttys002
[saraalbassam23@sarahs-MacBook-Air-9 ~ % edge-impulse-daemon --version
Edge Impulse serial daemon v1.31.0
Endpoints:
    Websocket: wss://remote-mgmt.edgeimpulse.com
    API:       https://studio.edgeimpulse.com
    Ingestion: https://ingestion.edgeimpulse.com
```

Figure 5-11 *Edge Impulse CLI.*

To run the model directly on the Nicla Vision board, we used the Edge Impulse CLI to connect our device and flash the model onto it. First, we physically connected the Nicla Vision camera to our computer using a USB cable. Then, we downloaded the official Nicla Vision firmware from Edge Impulse at: https://cdn.edgeimpulse.com/firmware/arduino-nicla-vision.zip.

Once this process was completed, the firmware was successfully installed on the device as shown in figure5-12.

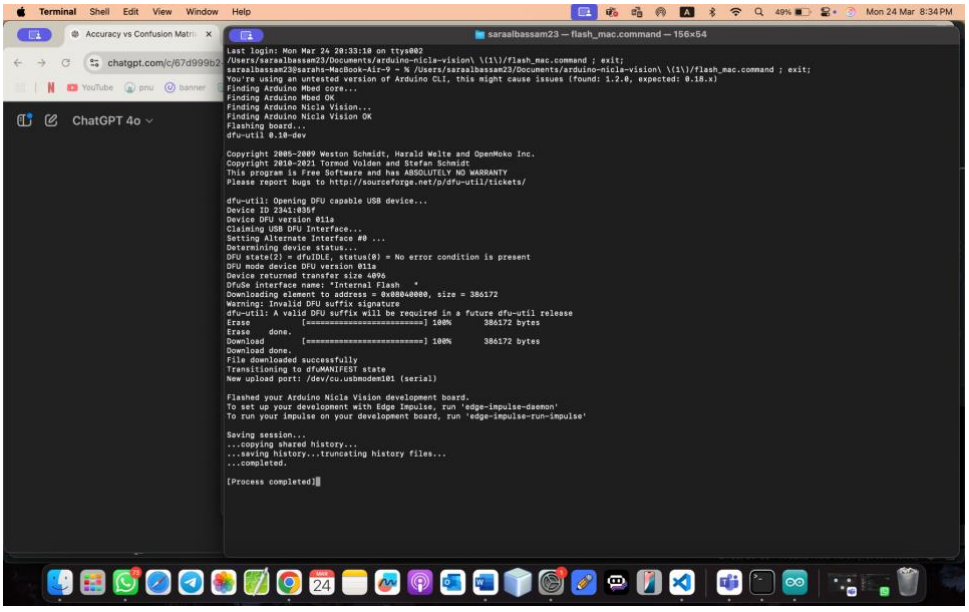


Figure 5-12: *Nicla vision firmware.*

To connect the camera to our Edge Impulse project, we opened a terminal and ran the command: *edge-impulse-daemon*

We entered our Edge Impulse login credentials and then selected our project from the list. After that, the Nicla Vision camera appeared under the Devices tab in Edge Impulse Studio with a green indicator, confirming that it was connected and ready. As shown in figure5-13.

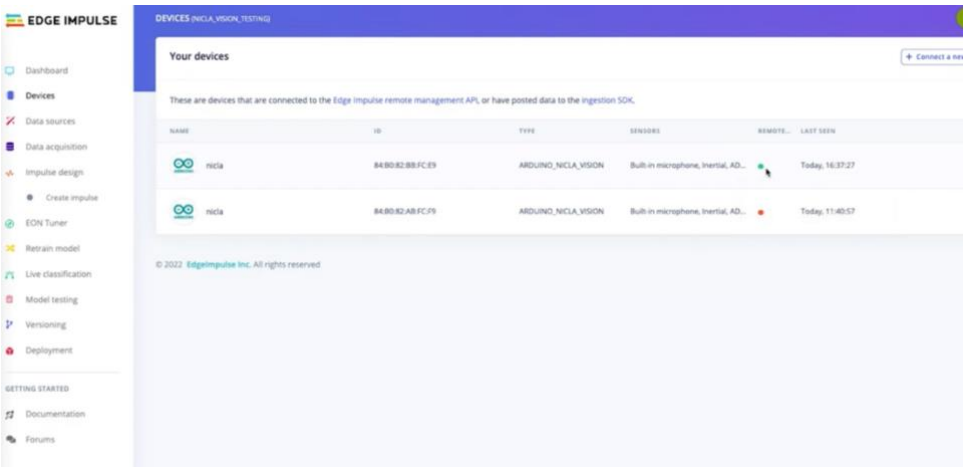


Figure 5-13: *Camera connection to the edge impulse.*

The final step was flashing the model to the Nicla Vision board. From within Edge Impulse, we built the model in a format compatible with Nicla Vision. As shown in figure5-14.

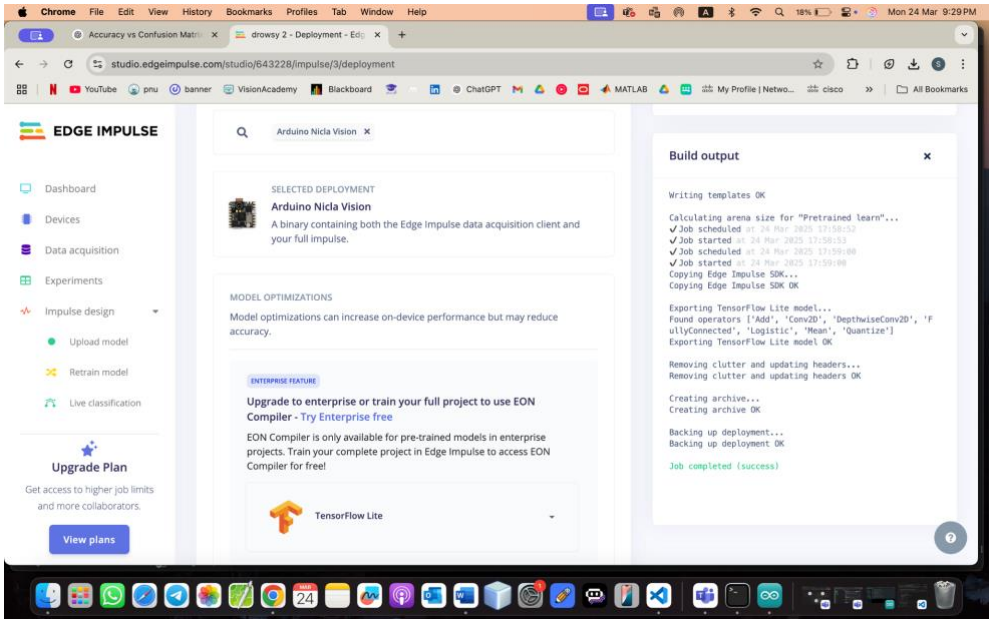


Figure 5-14: *Convert the model format.*

Then, we ran the following command in the terminal: *edge-impulse-run-impulse*

This command successfully uploaded the model to the device. After flashing, the Nicla Vision was able to perform real-time drowsiness detection, using its onboard camera and the AI model without requiring any internet connection or external

processing. With using Edge Impulse CLI, Figure5-15 shows running the model on Nicla Vision and figure5-16 shows the Real-Time inference output from Nicla Vision.

Figure 5-15: *Running the AI Model.*

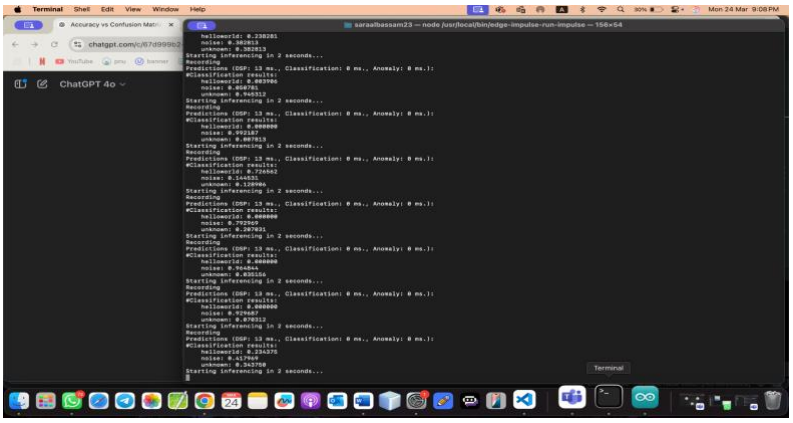


Figure5-16: *The Output result.*

5.4 Arduino IDE Configuration

To begin working with the Nicla Vision and integrate the buzzer functionality, we first installed the Arduino IDE—the official development environment for programming Arduino-compatible boards. The software provides all the essential tools for writing, verifying, and uploading code to the hardware. We downloaded the Arduino IDE from the official website and followed the installation instructions for our operating system. Figures5-17 and 5-18 illustrate the installation process and show the Arduino IDE interface once it's launched.

Figure 5-17*: Arduino IDE installation.*

Figure 5-18: *Arduino IDE Interface.*

After installation, we proceeded to install the necessary board support packages and libraries. Specifically, we added support for the Nicla Vision board and included the Edge Impulse Arduino library that we had previously exported. This allowed us to easily call our AI model within the Arduino environment and write the logic for real-time drowsiness detection.

5.4.1 Arduino Code

The Arduino IDE is a complete development platform. It includes a code editor, a message console for compiling and uploading status, and toolbars with buttons to verify, upload, and monitor serial outputs. It also features menus for managing libraries, boards, and port connections.

In our project, we wrote the Arduino code to interact with the Nicla Vision board. This code reads the model output and triggers a buzzer alert if drowsiness is detected. After writing the code, we selected the Nicla Vision board from the board manager and chose the correct COM port. We then uploaded the program directly to the Nicla Vision board. Once deployed, the board could analyze input from its built-in camera and activating the buzzer when the model detected signs of eye closure or drowsiness. The code snippet shown in Figure 5-19 represents the initial configuration for the drowsiness detection system. The BUZZER_PIN is set to pin 3, which is connected to the buzzer used for alerts. The variable drowsyCount is initialized to zero and is used to count the number of consecutive frames in which the driver appears drowsy. A threshold of 30 frames, defined by thresholdFrames, is used to determine when the buzzer should be activated. Additionally, the ALIGN_PTR macro is included for memory alignment purposes, which is useful in optimizing performance on embedded systems.

```
#define ALIGN_PTR(p,a)   ((p & (a-1)) ?(((uintptr_t)p + a) & ~(uintptr_t)(a-1)) : p)
#define BUZZER_PIN 3
int drowsyCount = 0;
const int thresholdFrames = 30;
```

Figure 5-19 *Initial Configuration of Buzzer Pin and Detection Parameters*

As shown in Figure 5-20, the buzzer pin is initialized in the setup() function of the Arduino code. The Serial.println() function displays a startup message for debugging purposes. The pinMode(BUZZER_PIN, OUTPUT); line sets the buzzer pin (defined earlier as pin 3) as an output pin. The line digitalWrite(BUZZER_PIN, LOW); ensures the buzzer is turned off at startup, preventing it from being triggered unintentionally when the system is powered on.

```
    Serial.println("Edge Impulse Inferencing Demo");
    pinMode(BUZZER_PIN, OUTPUT);
    digitalWrite(BUZZER_PIN, LOW);

```

Figure 5-20 *Buzzer Pin Setup in the Arduino setup() Function*

As shown in Figure 5-21, this section of the code implements the core drowsiness detection logic. If the prediction result equals "drowsy", the drowsyCount is incremented. If a non-drowsy state is detected, the counter resets to zero. Once the count reaches the thresholdFrames value (i.e., 30 frames), the system triggers a buzzer alert by setting the buzzer pin to HIGH for 4 seconds using the delay(4000) function. After the alert, the pin is set back to LOW to stop the buzzer, and the counter is reset to begin a new detection cycle.

```

    if (prediction == "drowsy") {
        drowsyCount++;
    } else {
        drowsyCount = 0;
    }

    if (drowsyCount >= thresholdFrames) {
        ei_printf("Drowsiness detected! Beeping buzzer for 4 seconds.\r\n");
        digitalWrite(BUZZER_PIN, HIGH);
        delay(4000);
        digitalWrite(BUZZER_PIN, LOW);
        drowsyCount = 0;
    }
```

Figure 5-21 *Drowsiness Detection Logic and Buzzer Activation Code*

5.5 The Proposed Architecture

The figure 5-22 shows the final proposed architecture.

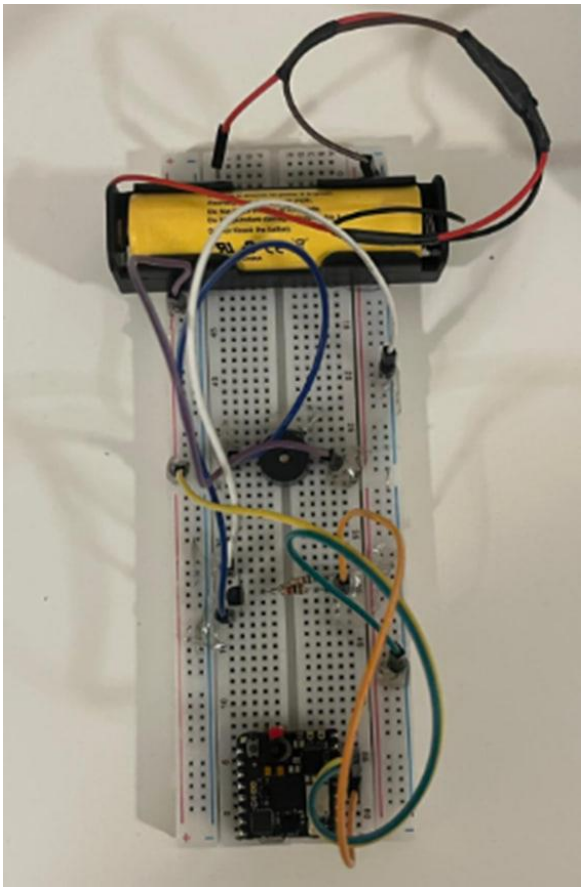


Figure 5-22 *The proposed architecture.*

Chapter 6: Testing

6.1 Test Plan

6.2 Test cases

6.2.1 Case 1 – System Initialization and Model Uploading

6.2.2 Case 2 - Normal Eye Blinking

6.2.3 Case 3 - Drowsiness Detection in Different Lighting Conditions

6.2.4 Case 4 – Detection When No Face in Frame

6.3 Test Results

6.3.1 Performance Metrics

6.3.1.1 Confusion Matrix

6.3.1.2 ROC Curve

6.3.2 Binary classification

6. Testing

6.1 Test Plan

The goal of this project is to develop a system that detects signs of driver drowsiness using a camera and alerts the driver in real time. The system analyzes continuously captured frames to classify the driver's state as either Drowsy or Not Drowsy.

To achieve this, a pretrained MobileNetV2 deep learning model was used for binary image classification. The input data consisted of labeled images collected from various individuals, representing the two target classes. The dataset was divided into 80% for training and 20% for testing to ensure a balanced evaluation of the model's performance.

The testing process aimed to assess how accurately and reliably the model could detect drowsiness. The evaluation was based on several performance metrics, including accuracy, sensitivity, specificity, precision, false positive rate, false negative rate, AUC (Area Under the Curve), Matthews Correlation Coefficient (MCC), and F1-score. These metrics provide a comprehensive view of the model's strengths and weaknesses in binary classification.

The model was trained and tested using Visual Studio Code (VSCode) as the development environment. One main experiment was conducted using 100 training epochs, and model performance was analyzed across different points in training to observe improvement trends.

To validate the system's behavior in real-world scenarios, four test cases were conducted as part of the testing process:

Case 1 – System Initialization and Model Uploading: Verifies that the system initializes properly, and the trained model is correctly loaded onto the Nicla Vision board without errors.

Case 2 – Normal Eye Blinking: Ensures that brief, natural eye blinks do not trigger false drowsiness alerts.

Case 3 – Drowsiness Detection in Different Lighting Conditions: Tests the model's robustness by evaluating detection performance under various lighting environments (e.g., dim, bright).

Case 4 – Detection When No Face in Frame: Confirms that the system handles edge cases gracefully and does not falsely detect drowsiness when no face is visible. This test plan ensures that the system's detection logic is thoroughly validated before integrating it into a real-time driver monitoring application.

6.2 Test cases

6.2.1 Case 1 – System Initialization and Model Uploading

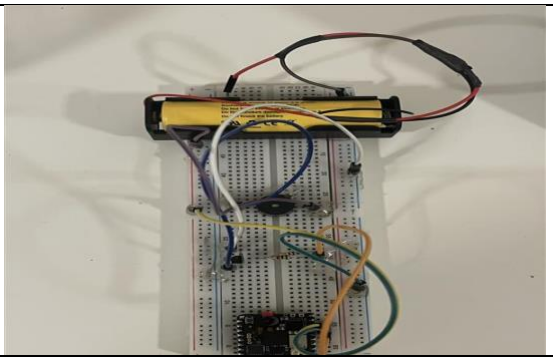
Test Objective	Ensure the system boots and the model loads successfully
Steps	1.connect nicla vison camera 2.load model into the camera
Expected Result	No error during intilazation , model loaded correctly
Actual Result	Model loaded successfully
Status	Passed
Trials	One
Visualization	

Table 6-1: *Case 1 – System Initialization and Model Uploading.*

6.2.2 Case 2 - Normal Eye Blinking

Test Objective	Ensure the system does not trigger an alarm when eyes just blink.
Steps	1. Blink naturally 2. Repeat over 10 trials 3. Observe buzzer response
Expected Result	No buzzer triggered during short blinks
Actual Result	Buzzer remained off in 9/10 trials
Status	Passed

Trials	10 trials
Visualization	

Table 6-2: *Case 2 - Normal Eye Blinking.*

6.2.3 Case 3 - Drowsiness Detection in Different Lighting Conditions

Test Objective	Ensure drowsiness detection works under both bright and dim lighting.
Steps	1. Close eyes under each lighting condition (light,dim). 2. Observe. 3. wait for buzzer response
Expected Result	Buzzer triggers in both lighting conditions
Actual Result	4/5 (bright), 3/5 (dim)
Accuracy	Bright: 80%, Dim: 60%
Status	Passed
Trials	10 trials
Visualization	

Table 6-3: *Case 3 - Drowsiness Detection in Different Lighting Conditions.*

6.2.4 Case 4 – Detection When No Face in Frame

Test Objective	Ensure the system does not trigger false alarms with no face visible Ensure.
Steps	1. Remove face from camera frame 2. Observe model output
Expected Result	No detection, no buzzer triggered
Actual Result	10/10 correct behavior
Accuracy	100%
Status	Passed
Trials	10 trials

Table 6-4: *Case 4 – Detection When No Face in Frame*

6.3 Test Result

6.3.1 Performance Metrics

To accurately assess the performance of the drowsiness detection model, a variety of evaluation metrics were utilized. These metrics are essential in understanding how well the model can classify input images into two categories: Drowsy and Not Drowsy. The classification results are based on the following outcomes:

- **True Positives (TP):** The number of instances where the model correctly predicted the driver as Drowsy, and the actual label was also Drowsy.
- **True Negatives (TN):** The number of instances where the model correctly predicted the driver as Not Drowsy, and the actual label was also Not Drowsy.
- **False Positives (FP):** Cases where the model incorrectly predicted Drowsy when the actual state was Not Drowsy.
- **False Negatives (FN):** Cases where the model incorrectly predicted Not Drowsy when the actual state was Drowsy.

Based on these outcomes, several performance metrics were computed:

- **Accuracy:** Measures the overall correctness of the model by calculating the ratio of correctly predicted instances (both TP and TN) to the total number of predictions [28].
- **Precision:** Indicates how many of the predicted Drowsy instances were actually correct. It is especially important in minimizing false alarms in real-time detection systems [28].
- **Recall (Sensitivity):** Reflects the model's ability to detect actual Drowsy cases. A high recall means fewer missed detections, which is critical in driver safety applications [29].
- **F1-Score:** The harmonic mean of precision and recall, offering a single balanced metric that accounts for both false positives and false negatives [29].
- **Specificity:** Measures how effectively the model identifies Not Drowsy cases, helping to reduce unnecessary alerts [30].
- **False Positive Rate (FPR):** The proportion of Not Drowsy instances that were incorrectly classified as Drowsy [31].
- **False Negative Rate (FNR):** The proportion of Drowsy instances that were incorrectly classified as Not Drowsy, which is crucial in avoiding missed warnings [31].
- **Matthews Correlation Coefficient (MCC):** A robust metric that considers TP, TN, FP, and FN, offering a balanced evaluation even when the dataset is imbalanced.
- **Area Under the Curve (AUC):** Represents the area under the Receiver Operating Characteristic (ROC) curve. It quantifies the model's ability to distinguish between the two classes across all thresholds. A higher AUC indicates better classification performance [29].
- These metrics collectively provide a comprehensive evaluation of the drowsiness detection system, ensuring that the model is both reliable and effective before deployment in real-time monitoring scenarios.

The following equations numbered from 1 to 9 illustrate the formulas of calculating each performance metric.

$$\text{Accuracy (\%)} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{1}$$

$$\text{Precision(\%)} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2}$$

$$\text{Recall}(\%) = \frac{\text{TPos}}{\text{TP} + \text{FN}} \tag{3}$$

$$\text{F1} - \text{Score}(\%) = 2 \times (\frac{\text{Recall} \times \text{Prec}}{\text{Recall} + \text{Prec}}) \tag{4}$$

$$\text{Sensitivity}(\%) = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{5}$$

$$\text{Specificity}(\%) = \frac{\text{TN}}{\text{TN} + \text{FP}} \tag{6}$$

$$\text{False positive rate}(\%) = 1 - \frac{\text{TN}}{\text{TN} + \text{FP}} \tag{7}$$

$$\text{False Negative rate}(\%) = 1 - \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{8}$$

$$\text{MCC}(\%) = \frac{\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}}{\sqrt{(\text{TP} + \text{FP}) \cdot (\text{TP} + \text{FN}) \cdot (\text{TN} + \text{FP}) \cdot (\text{TN} + \text{FN})}} \tag{9}$$

6.3.1.1 Confusion Matrix

The classification results are based on the following outcomes: The confusion matrix, also known as the error matrix, is a useful tool for visualizing the performance of a classification model on a given test dataset. It is widely used in classification problems, including both binary and multi-class scenarios. As the name suggests, a confusion matrix displays the instances where the model was confused in making its predictions. It shows how well a classification model performs in terms of correctly or incorrectly predicting each class. In simple terms, it connects the predicted outcomes with the actual labels of the data, making it easier to analyze where the model is performing well and where it is making errors. Since the confusion matrix relies on known target labels, it is only applicable in supervised learning tasks. This matrix not only helps in calculating the overall accuracy of a classifier but also enables the computation of several other essential performance metrics, including precision, recall, F1-score, and more [32]. It is built upon four key components:

- True Positives (TP)
- True Negatives (TN)
- False Positives (FP)
- False Negatives (FN)

These values allow for a deeper understanding of how the model is performing on both the positive and negative classes, and how balanced or biased predictions are.

Furthermore, confusion matrices can be used to compare multiple classifiers when they are evaluated on the same dataset. This comparison helps in identifying each model's strengths and weaknesses and can guide the design of ensemble learning techniques to combine models for better performance as shown in figure 6-1.

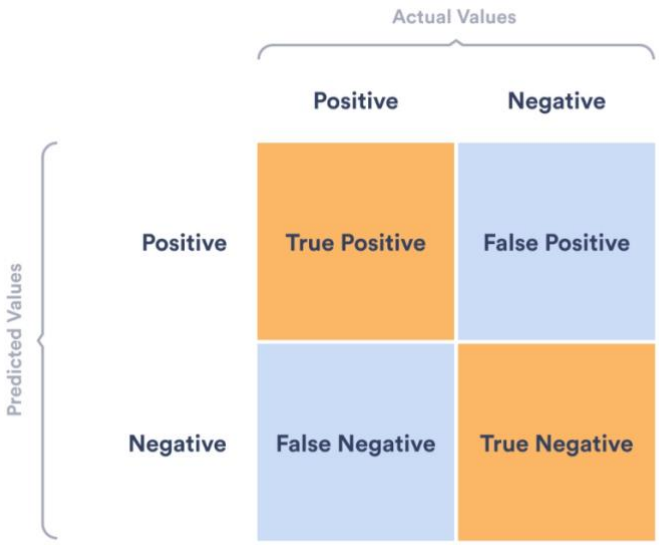


Figure6-1: *Showing Confusion Matrix in ML Binary.*

6.3.1.2 ROC Curve

A Receiver Operating Characteristic (ROC) curve is a graphical representation used to evaluate the performance of a binary classification model. It plots the True Positive Rate (TPR), also known as sensitivity, against the False Positive Rate (FPR) at various threshold levels. The ROC curve provides valuable insights into the trade-off between a model's sensitivity and specificity, allowing for a comprehensive assessment of classification performance across all thresholds. A key metric derived from the ROC curve is the Area Under the Curve (AUC). The AUC quantifies the overall ability of the model to distinguish between the two classes. An AUC of 1.0 represents a perfect classifier, while an AUC of 0.5 indicates performance equivalent to random guessing. Generally, a model with an AUC closer to 1.0 is considered highly effective, whereas values near 0.5 suggest poor classification ability.In addition to performance evaluation, ROC curves are also useful for comparing

different classifiers. When multiple models are tested on the same dataset, the one with the highest AUC is typically considered the best-performing model [33].

6.3.2 Binary classification

The evaluation results of the drowsiness detection model across different training epochs are presented in Table 6-1. The results demonstrate a clear improvement in the model's performance over time.

At epoch 10, the model achieved an accuracy of approximately 74.35%, with relatively high false positive (26.97%) and false negative (24.11%) rates. This indicates that, during the early training phase, the model struggled to accurately classify both drowsy and alert states.

By epoch 50, the model showed significant improvement. Accuracy increased to 92.87%, sensitivity (recall) reached 93.16%, and precision improved to 91.65%, highlighting the model's enhanced ability to correctly identify drowsy drivers while minimizing incorrect classifications. The false negative rate dropped to 6.84%, which is crucial for safety-critical applications like driver monitoring. Similarly, the false positive rate decreased to 7.38%, reducing unnecessary alerts.

The Area Under the Curve (AUC) steadily increased from 0.83 at epoch 10 to 0.98 at epoch 50, indicating a strong overall classification ability. Additionally, the Matthews Correlation Coefficient (MCC) improved from 0.49 to 0.86, showing better agreement between actual and predicted classes as training progressed.

Finally, the F1-score rose from 0.73 to 0.92, reflecting a balanced enhancement in both precision and recall. These results, as shown in Table 6-1, confirm the model's increasing reliability and effectiveness, making it suitable for real-time driver drowsiness detection.

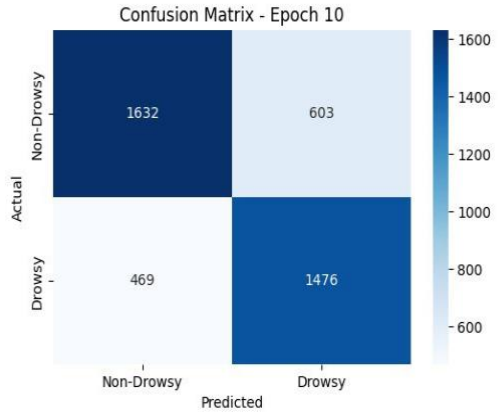
No of iterations	TP	TN	FP	FN	Accuracy	Sensitivity	Specificity	Precision	False Negative Rate	False Putative Rate	AUC	Matthews Correlation Coefficient	F1-score
10	1476	1632	603	469	74.35%	75.89%	73.02%	70.99%	24.11%	26.98%	0.82	0.4878	0.7335
20	1514	1858	377	431	80.67%	77.84%	83.13%	80.06%	22.16%	16.87%	0.89	0.6110	0.7893
30	1668	1948	287	277	86.51%	85.76%	87.16%	85.32%	14.24%	12.84%	0.93	0.7289	0.8553
40	1734	2008	227	211	89.52%	89.15%	89.84%	88.42%	10.85%	10.16%	0.96	0.7895	0.8878
50	1812	2070	165	133	92.87%	93.16%	92.62%	91.65%	6.84%	7.38%	0.97	0.8569	0.9240
100	1814	2143	92	131	94.67%	93.26%	95.88%	95.17%	6.74%	4.12%	0.98	0.8928	0.9420

Table 6-5: *Binary classification system for driver drowsiness detection in many iterations.*

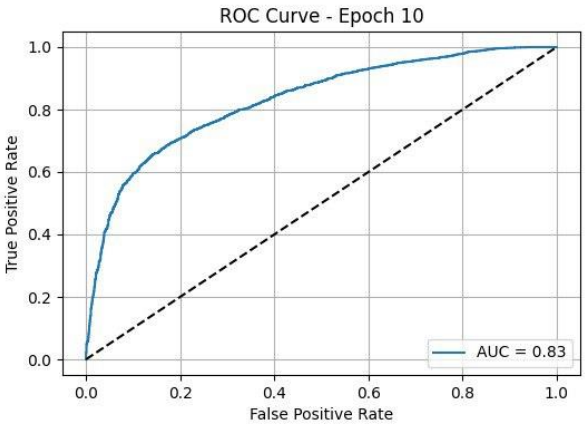
To further support the evaluation of the model's performance, visual representations of the confusion matrix and ROC curve are provided for selected training epochs. These

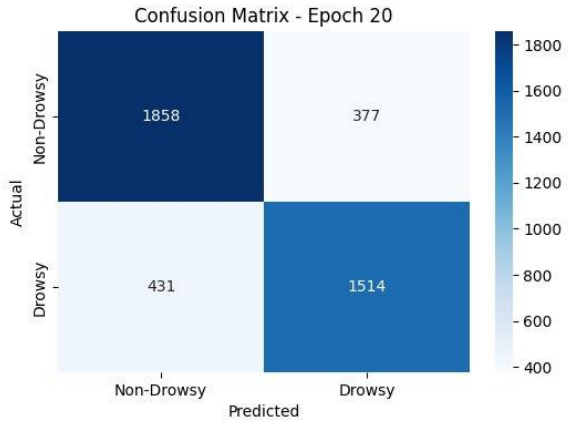
visualizations offer an intuitive understanding of how well the model distinguishes between the two classes—Drowsy and Not Drowsy—as training progresses.

The confusion matrix images highlight the classification accuracy by showing the distribution of true and false predictions, while the ROC curves illustrate the trade-off between the true positive rate and false positive rate across different threshold values. As seen in Table 6-2, both the confusion matrix and ROC curve evolve with training, reflecting the model's increasing ability to correctly identify the driver's state.

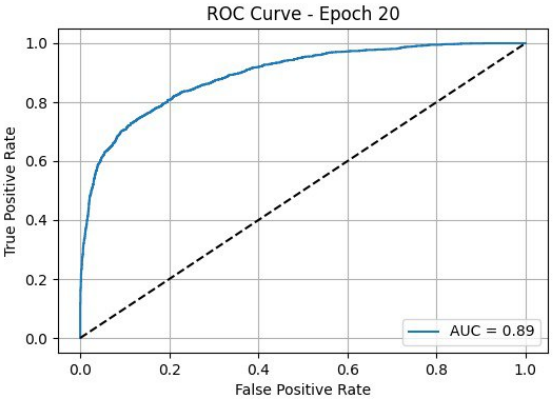


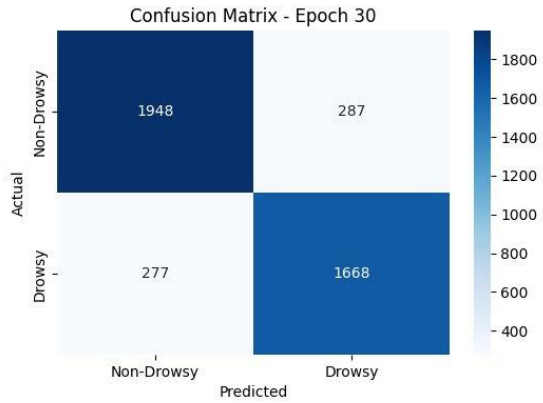
a) Confusion Matrix yielded from training of DL model for 10 iterations.



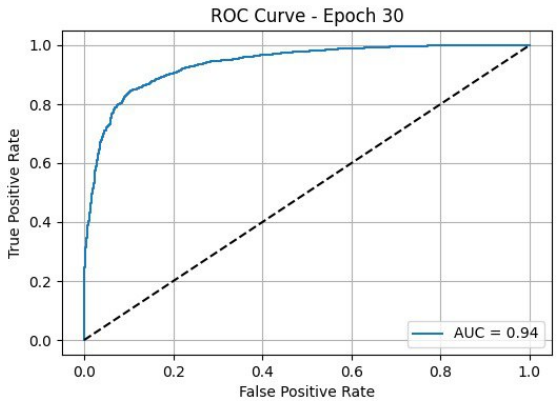


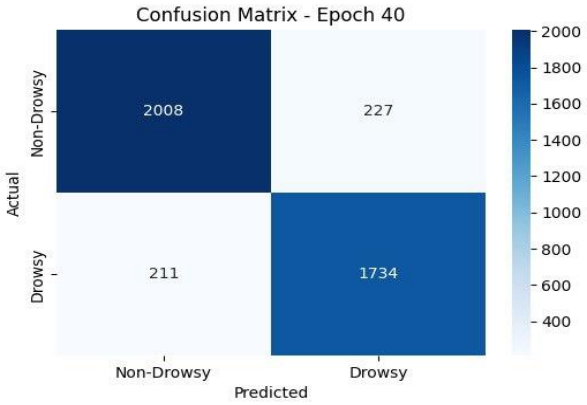
b) Confusion Matrix yielded from training of DL model for 20 iterations.



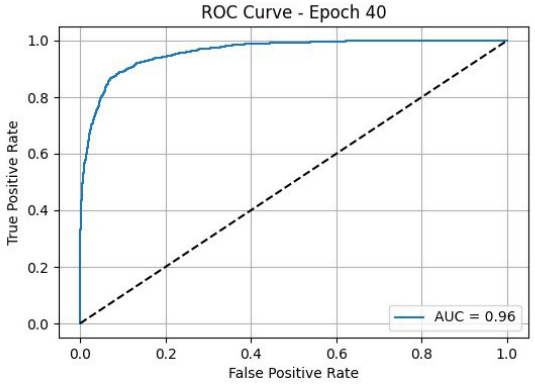


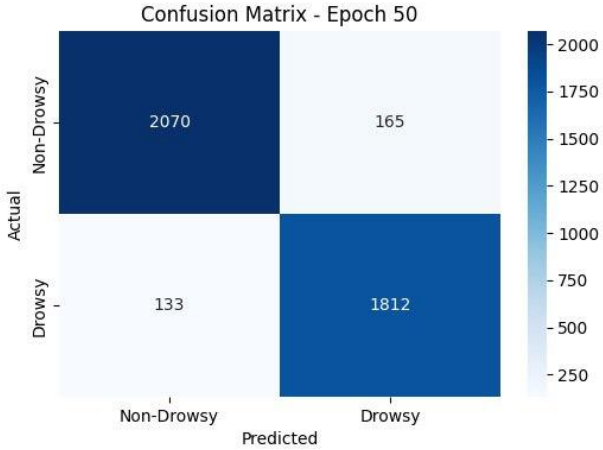
c) Confusion Matrix yielded from training of DL model for 30 iterations.



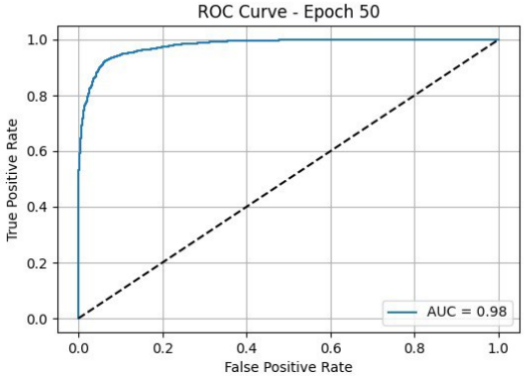


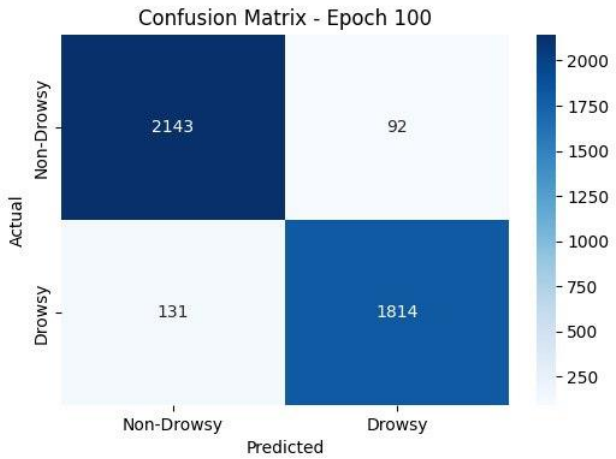
d) Confusion Matrix yielded from training of DL model for 40 iterations.





e) Confusion Matrix yielded from training of DL model for 50 iterations.





f) Confusion Matrix yielded from training of DL model for 100 iterations.

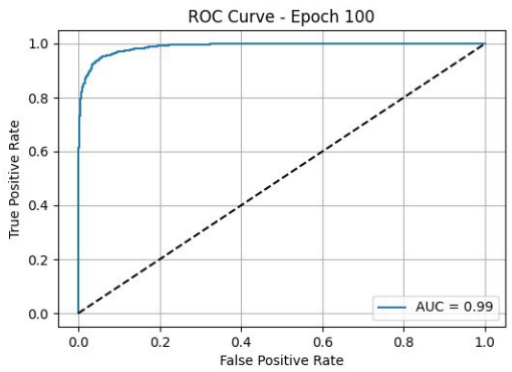


Table 6-6: *model's performance*

Chapter 7: Conclusion

---

7. Conclusion

7.1 Test cases Evaluation

7.2 Future Works

7. Conclusion

Our project successfully designed and implemented a real-time driver drowsiness detection system using the Arduino Nicla Vision Camera and a buzzer as a warning mechanism. The primary goal was to enhance road safety by identifying signs of fatigue in drivers and alerting them before accidents occur. Given that drowsy driving is a global concern responsible for thousands of road incidents annually, the system addresses a real and pressing issue.

At the heart of the implementation is the Arduino Nicla Vision, a compact and powerful AI edge device. It combines a high-resolution camera with onboard processing capabilities, making it ideal for real-time computer vision tasks without the need for constant cloud connectivity. Its ability to run TensorFlow Lite models directly on the device allows for low-latency, energy efficient perfect for embedded AI applications such as driver monitoring. Nicla Vision's small form factor and built-in wireless features further contribute to its flexibility and potential for integration into real-world automotive environments.

The core of the system is a lightweight convolutional neural network (CNN) model based on MobileNetV2, a pre-trained model optimized for edge devices. The model was converted to TensorFlow Lite (TFLite) to be compatible with the Nicla Vision camera. When the driver's eyes remain closed for more than 3 seconds, the buzzer is triggered as an alert, and it will continuously alert the driver until their eyes open again. This setup ensures quick response to drowsiness and minimizes safety risks.

A dataset of over 41,000 images—extracted from video sources and labeled accordingly—was used to fine-tune the MobileNet model. This enabled the system to differentiate between alert and drowsy states with high accuracy. The entire system was tested and verified, showing strong performance under real-world conditions.

7.1 Evaluation

In this project, we used a binary classification model to accurately detect whether the driver was drowsy or non-drowsy based on eye status. After training and testing the model with more than 41,000 images, we observed clear improvements in performance across multiple training stages. As training progressed, the model became more reliable, showing a significant increase in accuracy and a decrease in false predictions.

By the end of training, the model reached a high accuracy of **92.87%**, with a precision of **91.65%** and recall of **93.16%**. These results proved that the system can effectively detect drowsiness and respond quickly by triggering the buzzer. The false negative rate, which is

very important in safety-critical applications, dropped to only **6.84%**, and the overall classification performance was confirmed by an AUC of **0.98** and an F1-score of **0.92**.
We are satisfied with the performance of our model, especially considering it runs on a small edge device like the Nicla Vision. The results confirm that our system is suitable for real-time use and can help reduce risks associated with drowsy driving.

7.2 Future Works

In the future, we can make the system even better by adding GPS tracking, so it can send alerts with the driver's location in serious situations. We also plan to improve the model by using a larger dataset and testing it with different drivers and lighting conditions to make it more reliable. To make it work for more people, we want the system to detect drowsiness even if the driver is wearing glasses. We also aim to run the system using a battery for more flexibility and ease of use. As the system develops, we will continue working on improving its accuracy and performance to ensure it works well in all situations.

References

[1] S. Saleem, "Risk assessment of road traffic accidents related to sleepiness during driving: a systematic review," *Eastern Mediterranean Health Journal*, vol. 28, no. 9, pp. 695–700, Sep. 2022, doi: 10.26719/EMHJ.22.055.

[2] A. S. Bahammam *et al.*, "Prevalence of sleep-related accidents among drivers in Saudi Arabia," *Ann Thorac Med*, vol. 9, doi: 10.4103/1817-1737.140138.

[3] "Tesla cameras will monitor driver awareness." Accessed: Sep. 17, 2024. [Online]. Available: https://www.bbc.com/news/technology-57283479

[4] F. Safarov, F. Akhmedov, A. B. Abdusalomov, R. Nasimov, and Y. I. Cho, "Real-Time Deep Learning-Based Drowsiness Detection: Leveraging Computer-Vision and Eye-Blink Analyses for Enhanced Road Safety," *Sensors*, vol. 23, no. 14, Jul. 2023, doi: 10.3390/s23146459.

[5] P. Grzesik and D. Mrozek, "Combining Machine Learning and Edge Computing: Opportunities, Challenges, Platforms, Frameworks, and Use Cases," Feb. 01, 2024, *Multidisciplinary Digital Publishing Institute (MDPI)*. doi: 10.3390/electronics13030640.

[6] O. Jagtap, S. Chaurasia, P. Choudhary, G. Buwade, M. Dhote, and U. B. Aher, "Driver Drowsiness Detection System using Arduino and Deep Learning," *International Journal of Innovations in Engineering and Science*, vol. 8, no. 10, pp. 6–19, Nov. 2023, doi: 10.46335/ijies.2023.8.10.2.

[7] C. Collins, D. Dennehy, K. Conboy, and P. Mikalef, "Artificial intelligence in information systems research: A systematic literature review and research agenda," *Int J Inf Manage*, vol. 60, p. 102383, Oct. 2021, doi: 10.1016/J.IJINFOMGT.2021.102383.

[8] "Deep learning vs. machine learning." Accessed: Nov. 23, 2024. [Online]. Available: https://www.zendesk.com/blog/machine-learning-and-deep-learning/

[9] A. Abu Mallouh, Z. Qawaqneh, and B. D. Barkana, "Utilizing CNNs and transfer learning of pre-trained models for age range classification from unconstrained face images," *Image Vis Comput*, vol. 88, pp. 41–51, Aug. 2019, doi: 10.1016/J.IMAVIS.2019.05.001.

[10] "What Is Mobilenet V2? - GeeksforGeeks." Accessed: Nov. 23, 2024. [Online]. Available: https://www.geeksforgeeks.org/what-is-mobilenet-v2/

[11] K. Cao, Y. Liu, G. Meng, and Q. Sun, "An Overview on Edge Computing Research," *IEEE Access*, vol. 8, pp. 85714–85728, 2020, doi: 10.1109/ACCESS.2020.2991734.

[12] "What is Arduino Uno?" Accessed: Nov. 23, 2024. [Online]. Available: https://www.flyrobo.in/blog/what-is-arduino-uno?srsltid=AfmBOoqIDBePqR6_OnNcOz0AIdPZWPAg4cBoL2ffeauhMAB9kWYI2Ejz

[13] "Nicla Vision | Arduino Documentation." Accessed: Nov. 23, 2024. [Online]. Available: https://docs.arduino.cc/hardware/nicla-vision/#feature

[14] S. Giridhar et al., "An Intelligent System for Preventing Accidents Due to Driver Distractions," in Procedia Computer Science, Elsevier B.V., 2024, pp. 2196–2204. doi: 10.1016/j.procs.2024.04.208.

[15] M. Jain, B. Bhagerathi, and Dr. S. C N, "Real-Time Driver Drowsiness Detection using Computer Vision," Int J Eng Adv Technol, vol. 11, no. 1, pp. 109–113, Oct. 2021, doi: 10.35940/ijeat.A3159.1011121.

[16] Z. Ren et al., "EEG-Based Driving Fatigue Detection Using a Two-Level Learning Hierarchy Radial Basis Function," Front Neurorobot, vol. 15, Feb. 2021, doi: 10.3389/fnbot.2021.618408

[17] S. Ali Asim, "Driver Drowsiness Detection," 2023, doi: 10.55041/IJSREM22178.

[18] "Nicla Vision | Arduino Documentation." Accessed: Oct. 15, 2024. [Online]. Available: https://docs.arduino.cc/hardware/nicla-vision/?utm_source=chatgpt.com

[19] "Overview of the Arduino IDE 1 | Arduino Documentation." Accessed: Oct. 15, 2024. [Online]. Available: https://docs.arduino.cc/software/ide-v1/tutorials/Environment/

[20] "Getting Started with Nicla Vision | Arduino Documentation." Accessed: Oct. 15, 2024. [Online]. Available: https://docs.arduino.cc/tutorials/nicla-vision/getting-started/

[21] "What is Edge Impulse? | Edge Impulse Documentation." Accessed: Oct. 16, 2024. [Online]. Available: https://docs.edgeimpulse.com/docs/concepts/edge-ai/what-is-edge-impulse

[22] "What is edge machine learning (edge ML)? | Edge Impulse Documentation." Accessed: Oct. 16, 2024. [Online]. Available: https://docs.edgeimpulse.com/docs/concepts/what-is-edge-machine-learning

[23] "Arduino Nicla Vision | Edge Impulse Documentation." Accessed: Oct. 16, 2024. [Online]. Available: https://docs.edgeimpulse.com/docs/edge-ai-hardware/mcu/arduino-nicla-vision

[24] "Getting started | Edge Impulse Documentation." Accessed: Oct. 16, 2024. [Online]. Available: https://docs.edgeimpulse.com/docs

[25] Microsoft, "User Interface - Visual Studio Code," Visual Studio Code Documentation, Apr. 2025. [Online]. Available: https://code.visualstudio.com/docs/getstarted/userinterface

[26] N. Parmar and P. Hiscocks, "Drowsy Driver Detection System Design Project," 2002.

[27] Nasri, I. (2022). *Driver Drowsiness Dataset (DDD)*. Kaggle.com. https://www.kaggle.com/datasets/ismailnasri20/driver-drowsiness-dataset-ddd

[28] "The Difference between Accuracy and Precision Measurements | Precisa." Accessed: Nov. 21, 2023. [Online]. Available:https://www.precisa.com/blog/what-is-the-difference-between-accuracy-and-precision-measurements/

[29] "Precision and Recall in Classification Models | Built In." Accessed: Nov. 21, 2023. [Online]. Available: https://builtin.com/data-science/precision-and-recall

[30] R. Parikh, A. Mathai, S. Parikh, G. C. Sekhar, and R. Thomas, "Understanding and using sensitivity,

specificity and predictive values," Indian J Ophthalmol, vol. 56, no. 1, p. 45, 2008, doi: 10.4103/0301-4738.37595.

[31] "roc - Given true positive, false negative rates, can you calculate false positive, true negative? - Cross Validated." Accessed: Nov. 21, 2023. [Online]. Available: https://stats.stackexchange.com/questions/61829/given-true-positive-false-negative-rates-can-you-calculate-false-positive-tru

[32] "Confusion Matrix: How To Use It & Interpret Results [Examples]." Accessed: Nov. 21, 2023. [Online]. Available: https://www.v7labs.com/blog/confusion-matrix-guide

[33] "What is a ROC Curve and its usage in Performance Modelling?" Accessed: Nov. 21, 2023. [Online]. Available: https://www.tutorialspoint.com/what-is-a-roc-curve-and-its-usage-in-performance-modelling

Appendices

Appendix A: Survey Questions

1) How often do you drive for long durations (over 45 minutes) without taking a break?
2) Have you ever felt drowsy while driving?
3) If you felt drowsy while driving, what did you do?
4) Have you ever experienced an accident or near miss while driving due to drowsiness?
5) If the system detects signs of drowsiness, how would you prefer to receive an alert?
6) How often would you prefer the system to check for signs of drowsiness?
7) Would you feel safer if your car had a system that could detect drowsiness and alert you?
8) How important is it that the system works effectively in different lighting conditions (daytime, nighttime, bright sunlight)?
9) Do you think receiving false alarms from the drowsiness detection system would affect your trust in it?
10) How likely are you to use a drowsiness detection system